

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ



THIẾT KẾ VLSI ET4340

Thiết kế bộ giải mã sử dụng thuật toán Viterbi

Giảng viên hướng dẫn: TS. Nguyễn Vũ Thắng

Mã lớp: 169259

Nhóm 2:

Phan Thanh Bình 20223683

Lại Ngọc Đăng 20223898

HÀ NỘI, 2026

Phân công công việc

Thành viên	Nội dung thực hiện	Đóng góp
Phan Thanh Bình	- Thiết kế và code RTL các khối extract_bit, branch_metric, add_comp_slc - Viết testbench cho từng khối, testbench cho toàn hệ thống - Layout mạch - Viết báo cáo	60%
Lại Ngọc Đăng	- Tìm hiểu tổng quan lý thuyết mã hóa/giải mã Viterbi. - Thiết kế và code RTL các khối memory, traceback - Layout mạch	40%

Bảng sheet ghi chép nội dung thực hiện và mức độ hoàn thành của từng thành viên theo từng tuần:

https://docs.google.com/spreadsheets/d/1_tET47EQ4IV4fcOr-zHeCfcTo4gj6qXKVEFWQdLSlfY/edit?usp=sharing

Github của đề tài:

https://github.com/binhkbb2004-hash/Viterbi_Decoder_16bit

Drive của đề tài:

<https://drive.google.com/drive/folders/1FC7XBYZT9OpYvKsxJ4X66nJe8ZcUrUAj?usp=sharing>

Lời cảm ơn

Lời đầu tiên, nhóm 2 chúng em xin được gửi lời cảm ơn sâu sắc tới thầy TS. Nguyễn Vũ Thắng, người đã trực tiếp hướng dẫn và định hướng cho chúng em trong suốt quá trình thực hiện đề tài này.

Nhờ sự chỉ dẫn của thầy, đặc biệt trong việc tháo gỡ các vướng mắc kỹ thuật như xây dựng specification, sử dụng các công cụ mô phỏng, ... là yếu tố then chốt giúp nhóm có thể hoàn thành đề tài này. Những kiến thức và kinh nghiệm thầy chia sẻ không chỉ giúp nhóm có thể hoàn thiện sản phẩm hiện tại mà còn giúp chúng em rèn luyện thêm các tư duy logic, tư duy nhiên cứu, làm việc và phối hợp nhóm.

Dù đã cố gắng hết mình nhưng chắc chắn sản phẩm của nhóm 2 khó tránh khỏi những thiếu sót, chúng em rất mong sẽ nhận được những ý kiến đánh giá quý báu từ thầy để nhóm có thể hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

Tóm tắt nội dung đề tài

Đề tài này tập trung vào việc thiết kế và kiểm chứng phần cứng cho bộ giải mã Viterbi quyết định cứng cho mã chập với coding rate $R = 1/2$, constraint $K = 3$, đa thức sinh $x^2 + x + 1$ và $x^2 + 1$, chuyên dụng để giải mã chuỗi dữ liệu 16-bit. Đây là một khối xử lý tín hiệu lỗi đóng vai trò quan trọng trong các hệ thống truyền thông kỹ thuật số, giúp sửa lỗi và khôi phục dữ liệu một cách hiệu quả từ các kênh truyền nhiễu.

Hệ thống được mô tả phần cứng ở mức RTL sử dụng ngôn ngữ Verilog. Thiết kế áp dụng kiến trúc Pipelined Dataflow với 5 khối chức năng: `extract_bit`, `branch_metric`, `add_comp_slt`, `memory` và `traceback`.

Nhằm đảm bảo độ tin cậy tuyệt đối trước khi tiến hành thiết kế vật lý, quy trình kiểm chứng chức năng được thực hiện thông qua công cụ mô phỏng QuestaSim. Với kịch bản tự động hóa kiểm tra toàn bộ 1025 trường hợp dữ liệu mẫu, thiết kế đã đạt 1025/1025 testcase, đánh giá mức Code Coverage lý tưởng: 100% Statement và 100% Branch cho module `viterbi_top`. Kết quả này khẳng định logic giải mã hoạt động hoàn toàn chính xác.

Đồng thời, thiết kế đã được triển khai quá trình tổng hợp logic (Synthesis), thiết kế vật lý (Layout) bằng công cụ Cadence Genus và Cadence Innovus. Quá trình này bao gồm các bước: tổng hợp, floorplan, routing, check DRC và đánh giá các chỉ số PPA. Kết quả cho thấy hệ thống hoạt động ổn định ở tần số 200MHz, xuất thành công bản vẽ GDSII, sẵn sàng cho quá trình Tape – out

MỤC LỤC

Mục lục

CHƯƠNG 1. TỔNG QUAN THUẬT TOÁN GIẢI MÃ VITERBI	1
1.1 Tổng quan về Mã tích chập và Thuật toán Viterbi.....	1
1.1.1 Mã tích chập.....	1
1.1.2 Nguyên lí cơ bản của Thuật toán giải mã Viterbi.....	2
1.2 Phân tích cấu trúc Bộ mã hóa chập	2
1.3 Nguyên lí giải mã Viterbi	5
1.3.1 Sơ đồ lưới (Trellis Diagram)	5
1.3.2 Khoảng cách Hamming	5
1.3.3 Nguyên lí tìm đường đi tối ưu (Add – Compare – Select)	6
1.3.4 Quá trình Truy vết (Traceback)	7
1.4 Ví dụ minh họa	7
1.4.1 Quá trình mã hóa.....	7
1.4.2 Quá trình Giải mã	8
1.5 Ứng dụng của Thuật toán Viterbi	10
CHƯƠNG 2. THIẾT KẾ BỘ GIẢI MÃ VITERBI	11
2.1 Module viterbi_top	11
2.1.1 Sơ đồ khối	11
2.1.2 Chức năng	11
2.1.3 Hoạt động.....	12
2.1.4 Mô tả tín hiệu vào/ra (I/O).....	12
2.1.5 Waveform	13
2.2 Module extract_bit.....	14
2.2.1 Sơ đồ khối	14
2.2.2 Chức năng	14
2.2.3 Hoạt động.....	14
2.2.4 Mô tả tín hiệu vào/ra (I/O).....	15
2.2.5 Waveform	15
2.2.6 Lưu đồ thuật toán.....	16
2.3 Module branch_metric.....	17

2.3.1	Sơ đồ khối	17
2.3.2	Chức năng	17
2.3.3	Hoạt động.....	17
2.3.4	Mô tả tín hiệu vào/ra (I/O).....	18
2.3.5	Waveform	19
2.3.6	Lưu đồ thuật toán	19
2.4	Module add_comp_slt	20
2.4.1	Sơ đồ khối	20
2.4.2	Chức năng	20
2.4.3	Hoạt động.....	20
2.4.4	Mô tả tín hiệu vào/ra (I/O).....	22
2.4.5	Waveform	23
2.4.6	Lưu đồ thuật toán	24
2.5	Module memory	25
2.5.1	Sơ đồ khối	25
2.5.2	Chức năng	25
2.5.3	Hoạt động.....	25
2.5.4	Mô tả tín hiệu vào/ra (I/O).....	27
2.5.5	Waveform	28
2.5.6	Lưu đồ thuật toán	28
2.6	Module traceback	29
2.6.1	Sơ đồ khối	29
2.6.2	Chức năng	29
2.6.3	Hoạt động.....	29
2.6.4	Mô tả tín hiệu vào/ra (I/O).....	30
2.6.5	Waveform	31
2.6.6	Lưu đồ thuật toán	32
CHƯƠNG 3. KIỂM CHỨNG CHỨC NĂNG VÀ ĐÁNH GIÁ		33
3.1	Kiểm chứng ở mức Module (Sanity test)	33
3.1.1	Module extract_bit.....	33
3.1.2	Module branch_metric.....	34

3.1.3	Module add_comp_slr.....	34
3.1.4	Module memory.....	35
3.1.5	Module traceback.....	36
3.1.6	Module top.....	36
3.2	Kiểm chứng toàn hệ thống với Golden Model	37
3.2.1	Kiến trúc môi trường kiểm thử	37
3.2.2	Kịch bản mô phỏng và cơ chế Self – Checking.....	37
3.2.3	Kết quả mô phỏng tổng thể.....	38
3.3	Đánh giá chất lượng kiểm thử (Code Coverage).....	39
3.3.1	Các tiêu chí đánh giá.....	39
3.3.2	Đánh giá kết quả Coverage tổng thể.....	39
CHƯƠNG 4. THIẾT KẾ VẬT LÝ		41
4.1	Mục tiêu thiết kế	41
4.2	Tổng hợp logic (Logic Synthesis)	41
4.2.1	Thiết lập môi trường và ràng buộc	41
4.2.2	Kết quả tổng hợp.....	43
4.3	Thiết kế vật lý (Layout).....	45
4.3.1	Khởi tạo và Quy hoạch Floorplanning & Power Planning.....	45
4.3.2	Sắp xếp và tối ưu hóa sơ bộ (Placement & Pre-CTS)	46
4.3.3	Tổng hợp cây xung nhịp (Clock Tree Synthesis – CTS).....	46
4.3.4	Đi dây chi tiết và Tối ưu (Routing & Post – Route).....	47
4.3.5	Hoàn thiện và Kiểm tra vật lý (Sign-off & Physical Verification).....	47
4.4	Phân tích kết quả và đánh giá	49
4.4.1	Đánh giá Timing	49
4.4.2	Đánh giá Area	49
4.4.3	Đánh giá Power.....	50
4.5	Tổng kết.....	51
CHƯƠNG 5. KẾT LUẬN.....		53
5.1	Kết quả đạt được.....	53
5.2	Những hạn chế và bài học kinh nghiệm	54
5.3	Hướng phát triển của đề tài	54

TÀI LIỆU THAM KHẢO.....	56
PHỤ LỤC.....	57

DANH MỤC HÌNH VẼ

Hình 1.1: Shift Register ở bộ mã hóa.....	3
Hình 1.2: Sơ đồ chuyển trạng thái của bộ mã hóa	4
Hình 1.3: : Sơ đồ lưới Trellis cho mã (2,1,2).....	5
Hình 1.4: Sơ đồ mã hóa chập	7
Hình 2.1: Block Diagram Module top	11
Hình 2.2: Waveform module top	13
Hình 2.3: Block Diagram Module extract_bit	14
Hình 2.4: Waveform module extract_bit	15
Hình 2.5: Flowchart module extarct_bit	16
Hình 2.6: Block Diagram Module branch_metric	17
Hình 2.7: Waveform module branch_metric	19
Hình 2.8: Flowchart module branch_metric	19
Hình 2.9: Block Diagram Module ACS.....	20
Hình 2.10: Waveform module ACS.....	23
Hình 2.11: Flowchart module ACS.....	24
Hình 2.12: Block Diagram Module memory	25
Hình 2.13: Minh họa mảng nhớ module memory	26
Hình 2.14: Waveform module memory	28
Hình 2.15: Flowchart module memory	28
Hình 2.16: Block Diagram Module traceback	29
Hình 2.17: Waveform module traceback	31
Hình 2.18: Flowchart module traceback	32
Hình 3.1: Waveform mô phỏng module extract_bit	33
Hình 3.2: Waveform mô phỏng module branch_metric	34
Hình 3.3: Waveform mô phỏng module add_comp_slc.....	34
Hình 3.4: Waveform mô tả module memory	35
Hình 3.5: Waveform mô phỏng module traceback.....	36
Hình 3.6: Waveform mô phỏng module top	36
Hình 3.7: Kết quả trên phần mềm EE4235 Viterbi Online	37
Hình 3.8: Kết quả mô phỏng trên QuestaSim	38
Hình 3.9: Báo cáo Coverage trên QuestaSim	40
Hình 4.1: Nội dung file genus.tcl	42
Hình 4.2: Nội dung file constraints.sdc.....	43

Hình 4.3: Report Timing sau bước Synthesis	44
Hình 4.4: Report Area sau bước Synthesis	44
Hình 4.5: Report Power sau bước Synthesis.....	45
Hình 4.6: Mạch sau bước quy hoạch Floorplan và Powerplan	46
Hình 4.7: Kết quả check_drc.....	47
Hình 4.8: Kết quả check_connectiyity	48
Hình 4.9: Mạch sau khi hoàn tất Layout.....	48
Hình 4.10: Report Timing sau Layout	49
Hình 4.11: Report Area sau Layout	50
Hình 4.12: Report Power sau Layout.....	51

DANH MỤC BẢNG

Bảng 1.1: Bảng trạng thái cho mã $(n,k,m) = (2,1,2)$	4
Bảng 2.1: Mô tả tín hiệu I/O module top	12
Bảng 2.2: Mô tả tín hiệu I/O module extract_bit	15
Bảng 2.3: Mô tả tín hiệu I/O module branch_metric	18
Bảng 2.4: Mô tả tín hiệu I/O module ACS	22
Bảng 2.5: Mô tả tín hiệu I/O module memory	27
Bảng 2.6: Mô tả tín hiệu I/O module traceback	30
Bảng 4.1: Tổng hợp thông số PPA và kết quả Sign – off sau quá trình Layout	51

CHƯƠNG 1. TỔNG QUAN THUẬT TOÁN GIẢI MÃ VITERBI

Chương 1 trình bày cơ sở lý thuyết về mã tích chập và thuật toán giải mã Viterbi. Quá trình mã hóa dựa trên phép tích chập chuỗi thông tin và các đa thức sinh. Thuật toán Viterbi, dựa trên nguyên lý cực đại khả năng (Maximum Likelihood) được sử dụng để tìm chuỗi đầu vào có xác suất cao nhất thông qua việc so sánh các khoảng cách Hamming tích lũy. Nội dung chương là nền tảng cho việc thiết kế và mô phỏng các khối chức năng của bộ giải mã Viterbi.

1.1 Tổng quan về Mã tích chập và Thuật toán Viterbi

1.1.1 Mã tích chập

Trong các hệ thống truyền tin thực tế, tín hiệu truyền qua kênh vật lý luôn phải đối mặt với các loại nhiễu (noise) làm sai lệch các bit dữ liệu. Để giải quyết vấn đề này, Mã tích chập (Convolutional Code) là một trong những kỹ thuật mã hóa kênh được sử dụng phổ biến trong các hệ thống thông tin số nhằm phát hiện và sửa lỗi truyền. Khác với mã khối, trong đó dữ liệu được xử lý theo từng khối độc lập, mã tích chập xử lý chuỗi dữ liệu đầu vào liên tục và tạo ra chuỗi đầu ra phụ thuộc vào nhiều bit đầu vào trước đó. Cách thức này giúp tăng khả năng phát hiện lỗi và cải thiện độ tin cậy của hệ thống.

Một bộ mã hóa tích chập thường bao gồm các phần tử nhớ và các cổng logic XOR được kết nối theo cấu trúc xác định bởi các đa thức sinh. Tại mỗi chu kỳ xung clock, bộ mã hóa nhận vào k bit và xuất ra n bit, tạo ra mã có tốc độ $R = \frac{k}{n}$ với $R < 1$. Độ dài ràng buộc K thể hiện số phần tử nhớ của bộ mã hóa, tức là mức độ mà đầu ra hiện tại phụ thuộc vào các bit đầu vào trước đó.

Tập hợp các phép toán này có thể mô hình hóa bằng sơ đồ khối tương tự như một bộ lọc số trong xử lý tín hiệu rời rạc. Khi biểu diễn theo miền toán học, mối quan hệ giữa chuỗi đầu vào và đầu ra có thể được mô tả bằng phép tích chập:

$$V(t) = U(t) * G(t) \quad \text{PT 1-1}$$

Trong đó:

- $U(t)$: chuỗi dữ liệu đầu vào
- $G(t)$: hàm đáp ứng xung của bộ mã hóa
- $V(t)$: chuỗi đầu ra đã được mã hóa

Mã tích chập đặc biệt hiệu quả trong việc chống nhiễu ngẫu nhiên trong kênh truyền. Trong các hệ thống thực tế, mã tích chập được sử dụng kết hợp với thuật toán giải mã Viterbi, một thuật toán cực đại khả năng (Maximum Likelihood) nhằm xác định chuỗi bit đầu vào có xác suất cao nhất đã tạo ra chuỗi đầu ra nhận được. Nhờ đó, hệ thống có thể khôi phục dữ liệu gốc với độ tin cậy cao, ngay cả trong điều kiện kênh truyền có nhiễu mạnh hoặc suy hao tín hiệu.

1.1.2 Nguyên lý cơ bản của Thuật toán giải mã Viterbi

Để giải mã luồng dữ liệu của mã hóa chập, thuật toán Viterbi (được đề xuất năm 1967 bởi Andrew Viterbi) là phương pháp giải mã tối ưu nhất dựa trên nguyên lý Maximum Likelihood Decoding.

Thay vì đoán định từng bit một cách rời rạc, bản chất của Viterbi là một thuật toán Quy hoạch động (Dynamic Programming). Thuật toán dựa trên một sơ đồ lưới (Trellis Diagram) – đồ thị mô tả toàn bộ không gian các trạng thái và các ngã rẽ hợp lệ mà bộ mã hòa có thể sinh ra theo thời gian.

Khi nhận được một chuỗi bit (có thể chứa lỗi), thuật toán sẽ đối chiếu chuỗi này với các nhánh trên sơ đồ lưới để tính toán mức độ sai lệch. Tại mỗi nút trạng thái, thuật toán thực hiện cộng dồn các mức lỗi này, so sánh các ngã rẽ cùng hướng về một trạng thái và chỉ giữ lại nhánh có tổng số lỗi nhỏ nhất gọi là nhánh sống sót. Bằng cách loại bỏ dần các nhánh chứa nhiều lỗi, thuật toán sẽ tìm ra một “Đường sống sót” (Survivor Path) duy nhất xuyên đồ thị. Con đường này đại diện cho chuỗi trạng thái có xác suất xảy ra cao nhất, từ đó cho phép khôi phục lại chính xác chuỗi bit ban đầu.

Nhờ cấu trúc tính toán lặp lại có tính quy luật cao (Add – Compare – Select), thuật toán Viterbi đặc biệt phù hợp để triển khai và tối ưu hóa tốc độ trên các hệ thống phần cứng vi mạch.

1.2 Phân tích cấu trúc Bộ mã hóa chập

Để hiểu rõ cách thức giải mã, trước tiên cần phân tích chính xác kiến trúc và quy luật sinh mã của bộ mã hóa chập ở phía phát. Theo yêu cầu của đề tài, hệ thống mã hóa được thiết kế với các đặc tả kỹ thuật cụ thể như sau:

- Tỷ lệ mã hóa (Coding Rate): $R = 1/2 \Rightarrow$ ứng với mỗi 1 bit đầu vào bộ mã hóa sinh ra 2 bit đầu ra ($k = 1, n = 2$)
- Chiều dài ràng buộc (Constraint Length): $K = 3 \Rightarrow m = 2$. Thông số này chỉ ra rằng cặp bit đầu ra tại một thời điểm không chỉ phụ thuộc vào bit đầu vào hiện tại, mà còn phụ thuộc vào 2 bit đầu vào ở các chu kỳ trước đó. Về mặt phần cứng, hệ thống cần sử dụng một thanh ghi dịch (Shift Register) gồm 2 phần tử nhớ (tương đương với 2 Flip-Flop) để lưu trữ 2 trạng thái trước đó. Gọi 2 phần tử nhớ này là S_0 và S_1 . Do đó hệ thống sẽ có tổng cộng $N = 2^m = 2^2 = 4$ trạng thái có thể xảy ra: 00, 01, 10, 11

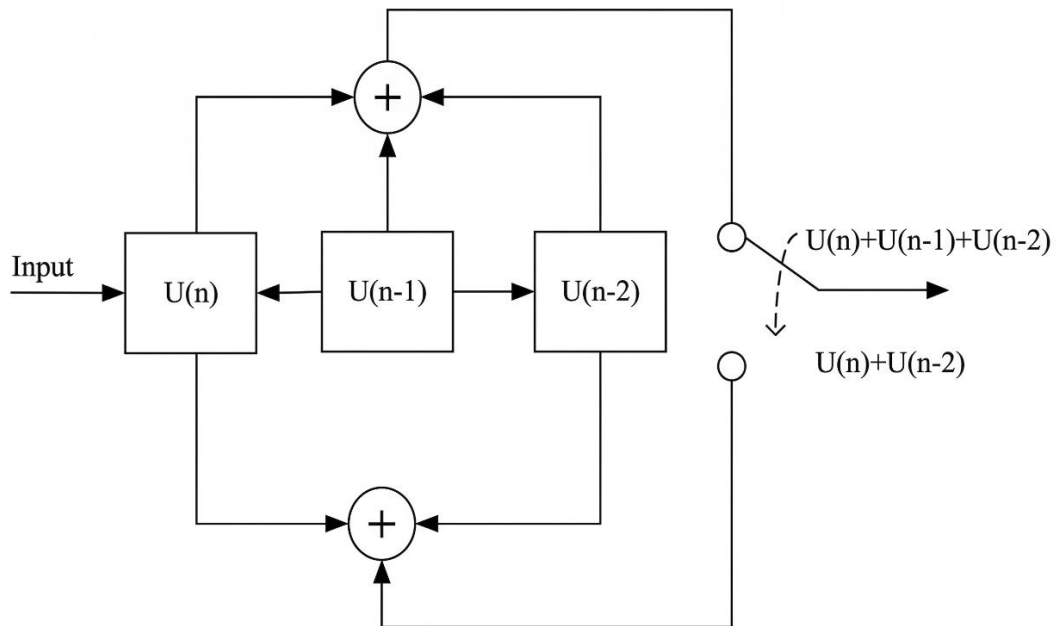
Sử dụng 2 đa thức sinh:

- $G_1 = x^2 + x + 1 \Rightarrow$ Đa thức này tương đương với cấu trúc toán học $U(n) + U(n-1) + U(n-2)$. Về mặt logic phần cứng, tín hiệu đầu ra thứ nhất (V_1) được tạo ra bằng cách XOR bit đầu vào hiện tại ($Input$) với 2 bit trạng thái S_0 và S_1 :

$$V_1 = Input \oplus S_0 \oplus S_1 \quad PT\ 1-2$$

- $G_2 = x^2 + 1 \Rightarrow$ Đa thức này tương đương với cấu trúc toán học $U(n) + U(n-2)$. Tín hiệu đầu ra thứ 2 (V_2) được tạo ra bằng cách XOR bit đầu vào hiện tại ($Input$) với bit trạng thái S_1 :

$$V_2 = Input \oplus S_1 \quad PT\ 1-3$$



Hình 1.1: Shift Register ở bộ mã hóa

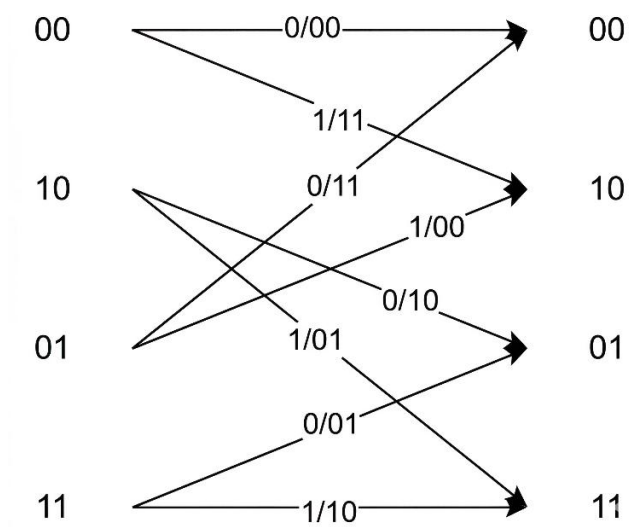
Nguyên lý hoạt động theo thời gian: Tại mỗi chu kỳ xung clock, sau khi V_1 và V_2 được tính toán xong, phần cứng sẽ thực hiện phép dịch bit. Giá trị $Input$ hiện tại sẽ được đẩy vào S_0 , giá trị cũ của S_0 bị đẩy sang S_1 và giá trị cũ của S_1 sẽ bị loại bỏ khỏi thanh ghi

Dựa vào phương trình logic của 2 đa thức sinh và nguyên lí dịch bit ở trên, ta có thể xây dựng bảng chuyển đổi trạng thái của toàn bộ hệ thống:

Bảng 1.1: Bảng trạng thái cho mã $(n,k,m) = (2,1,2)$

Input	Trạng thái hiện tại (S_0S_1)	Trạng thái kế tiếp (S_0S_1)	Output (V_1V_2)
0	00	00	00
0	01	00	11
0	10	01	10
0	11	01	01
1	00	10	11
1	01	10	00
1	10	11	01
1	11	11	10

Từ Bảng 1.1, ta xây dựng được sơ đồ chuyển trạng thái và của mã $(2,1,2)$:



Hình 1.2: Sơ đồ chuyển trạng thái của bộ mã hóa

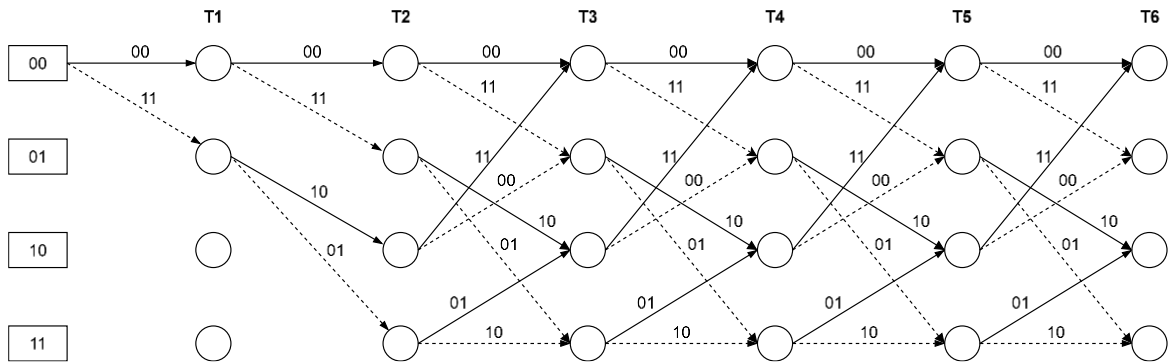
1.3 Nguyên lý giải mã Viterbi

Mục tiêu của thuật toán Viterbi tại đầu thu là đối chiếu chuỗi bit nhận được (có thể đã bị sai lệch do nhiễu) với quy luật mã hóa ban đầu để tìm ra chuỗi dữ liệu gốc có khả năng xảy ra cao nhất. Quá trình này được thực hiện thông qua việc duyệt trên một cấu trúc đồ thị gọi là Sơ đồ lưới Trellis

1.3.1 Sơ đồ lưới (Trellis Diagram)

Sơ đồ lưới là sự trải phẳng đồ thị chuyển trạng thái của bộ mã hóa (Hình 1.2) theo trục thời gian. Do bộ mã hóa có chiều dài ràng buộc $K = 3$, đồ thị Trellis sẽ có 4 trạng thái (00, 01, 10, 11) tại mỗi bước thời gian t_i . Dựa vào trạng thái khởi tạo mặc định 00, đồ thị Trellis chia làm 2 giai đoạn hoạt động rõ rệt:

- *Giai đoạn khởi động*: Diễn ra trong 2 chu kỳ thời gian đầu tiên ($t_0 \rightarrow t_2$). Tại t_0 , hệ thống chắc chắn ở trạng thái 00. Từ t_0 đến t_1 , đồ thị chỉ rẽ ra 2 nhánh đi đến trạng thái 00 và 10. Từ t_1 đến t_2 , từ 2 trạng thái này tiếp tục rẽ ra 4 nhánh đi tới cả 4 trạng thái. Đặc điểm cốt lõi của giai đoạn này là không có sự chồng chéo nhánh (mỗi trạng thái đích chỉ nhận đúng 1 đường đi vào).
- *Giai đoạn ổn định*: Bắt đầu từ thời điểm t_2 trở đi. Lúc này, cả 4 trạng thái đều đã được kích hoạt. Từ bất kỳ thời điểm t_n sang t_{n+1} ($n > 2$), mỗi trạng thái đích sẽ luôn nhận đúng 2 nhánh khác nhau đổ về từ 2 trạng thái ở bước trước đó. Sự hội tụ này đòi hỏi thuật toán phải có cơ chế chọn lọc để tránh bùng nổ tổ hợp đường đi.



Hình 1.3: : Sơ đồ lưới Trellis cho mã (2,1,2)

1.3.2 Khoảng cách Hamming

Để đánh giá mức độ sai lệch của dữ liệu thu được, thuật toán sử dụng phép đo Khoảng cách Hamming. Khoảng cách Hamming giữa 2 chuỗi bit có cùng chiều dài là số lượng các vị trí mà tại đó các bit tương ứng khác nhau.

Tại mỗi chặng thời gian (tương ứng 1 cặp bit đầu vào), bộ giải mã sẽ tính toán *Lỗi chặng (Branch Metric - BM)* bằng cách so sánh cặp bit thực tế nhận được với

các cặp bit đầu ra lý tưởng trên từng nhánh của sơ đồ Trellis. BM càng nhỏ, xác suất nhánh đó là con đường đúng càng cao.

1.3.3 Nguyên lý tìm đường đi tối ưu (Add – Compare – Select)

Đề đối phó với hiện tượng hội tụ nhánh trong *Giai đoạn ổn định* (khi mỗi trạng thái luôn nhận 2 đường đổ về), thuật toán áp dụng cơ chế Quy hoạch động. Về mặt toán học, quá trình cập nhật trạng thái này có thể được biểu diễn thông qua công thức tổng quát:

$$PM(S_j, t) = \min\{PM(S_{i1}, t-1) + BM(S_{i1} \rightarrow S_j), PM(S_{i2}, t-1) + BM(S_{i2} \rightarrow S_j)\} \quad PT\ 1-4$$

Trong đó:

- $PM(S_j, t)$: Tổng lỗi tích lũy (Path Metric) tại trạng thái đích S_j ở thời điểm t
- S_{i1} và S_{i2} : 2 trạng thái ở bước thời gian trước đó ($t-1$) có nhánh rẽ đi tới được trạng thái S_j
- $PM(S_{i1}, t-1)$ và $PM(S_{i2}, t-1)$: Tổng lỗi tích lũy đã được tính toán và lưu trữ của 2 trạng thái S_{i1} và S_{i2} tại thời điểm $t-1$
- $BM(S_{i1} \rightarrow S_j)$: Lỗi chằng (Branch Metric) của nhánh đi từ S_{i1} đến S_j . Giá trị này chính là khoảng cách Hamming giữa cặp bit thu được ở thời điểm t so với cặp bit lý tưởng sinh ra trên nhánh đó

Cách hệ thống phần cứng thực thi công thức trên:

- *Cộng (Add)*: Tính toán các biểu thức bên trong ngoặc. Hệ thống cộng giá trị Lỗi chằng (BM) của nhánh mới vào Tổng lỗi tích lũy (PM) cũ của trạng thái trước đó
- *So sánh (Compare)*: Thực hiện chức năng của hàm $\min$. Đưa 2 giá trị tổng lỗi vừa tính toán được vào một bộ so sánh (Comparator) để xác định giá trị nào nhỏ hơn
- *Chọn (Select)*: Bộ giải mã chỉ giữ lại nhánh có tổng lỗi nhỏ hơn (đường có ít lỗi hơn). Nhánh này được gọi là *Đường sống sót (Survivor Path)* và giá trị lỗi của nó được lưu lại thành $PM(S_j, t)$ mới. Nhánh có lỗi lớn hơn sẽ bị loại bỏ hoàn toàn khỏi bộ nhớ

Nhờ cơ chế ACS tuân thủ nghiêm ngặt công thức trên, tại bất kỳ thời điểm nào trong *Giai đoạn ổn định*, hệ thống cũng chỉ cần duy trì chính xác 4 Đường sống sót tương ứng với 4 trạng thái, giúp tối ưu hóa tài nguyên phần cứng và ngăn ngừa bùng nổ tổ hợp đường đi

1.3.4 Quá trình Truy vết (Traceback)

Khi toàn bộ chuỗi 16 bit mã hóa đã được đưa vào bộ giải mã và xử lý xong khối ACS, hệ thống sẽ tiến hành đánh giá 4 giá trị Path Metric cuối cùng tại thời điểm kết thúc.

Trạng thái có PM nhỏ nhất sẽ được chọn làm điểm xuất phát. Từ điểm này, bộ giải mã sẽ truy xuất lại bộ nhớ để lần ngược (Traceback) theo các quyết định "Chọn" của khối ACS trong quá khứ. Mỗi bước lùi về trước sẽ xác định chính xác một nhánh đã đi qua, từ đó ánh xạ ngược ra bit dữ liệu gốc ($Input$ là 0 hay 1). Kết quả của quá trình truy vết chính là chuỗi 8 bit thông tin ban đầu đã được khôi phục và loại bỏ nhiễu.

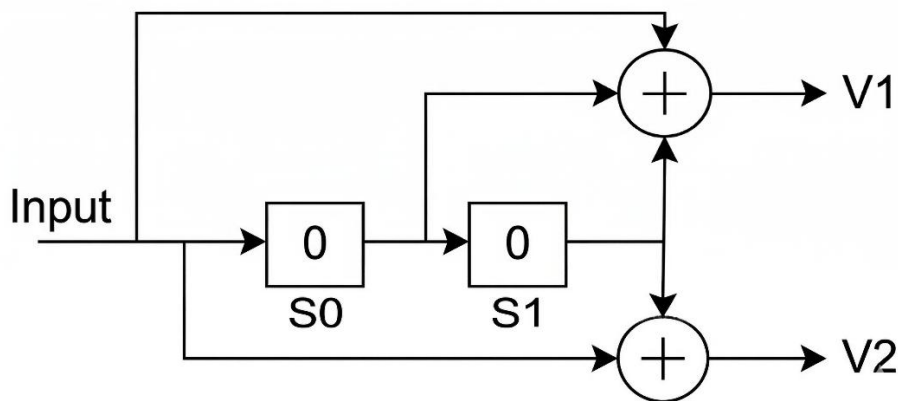
1.4 Ví dụ minh họa

Để làm rõ phương pháp luận của thuật toán Viterbi, phần này sẽ mô phỏng chi tiết quá trình mã hóa một bản tin 8 bit và quá trình giải mã khôi phục bản tin đó khi có xuất hiện nhiễu trên đường truyền.

Giả sử chuỗi 8 bit thông tin gốc cần truyền đi là: $1\ 1\ 0\ 0\ 1\ 0\ 1\ 0$ (Dữ liệu được đẩy vào hệ thống lần lượt từ trái sang phải)

1.4.1 Quá trình mã hóa

Xét sơ đồ mã hóa với 1 bit đầu vào, 2 bit đầu ra và sử dụng 2 thanh ghi dịch chứa 1 bit. Hệ thống mã hóa bắt đầu với trạng thái khởi tạo của 2 thanh ghi là $S_0 = 0, S_1 = 0$ (Trạng thái 00), áp dụng 2 đa thức sinh $G_1 = x^2 + x + 1$ và $G_2 = x^2 + 1$



Hình 1.4: Sơ đồ mã hóa chập

Quá trình sinh mã diễn ra theo từng chu kì xung nhịp như sau:

Ta có:

$$V_1 = Input \oplus S_0 \oplus S_1$$

$$V_2 = Input \oplus S_1$$

- Chu kì 1 ($Input = 1$): Trạng thái hiện tại: 00
 - $V_1 = 1 \oplus 0 \oplus 0 = 1, V_2 = 1 \oplus 0 \Rightarrow$ *Cặp bit*: 11
 - Trạng thái tiếp theo: Dịch bit $S_0 = 1, S_1 = 0$
- Chu kì 2 ($Input = 1$): Trạng thái hiện tại: 10
 - $V_1 = 1 \oplus 1 \oplus 0 = 0, V_2 = 1 \oplus 0 \Rightarrow$ *Cặp bit*: 01
 - Trạng thái tiếp theo: Dịch bit $S_0 = 1, S_1 = 1$
- Chu kì 3 ($Input = 0$): Trạng thái hiện tại: 11
 - $V_1 = 0 \oplus 1 \oplus 1 = 0, V_2 = 0 \oplus 1 \Rightarrow$ *Cặp bit*: 01
 - Trạng thái tiếp theo: Dịch bit $S_0 = 0, S_1 = 1$

Tiếp tục tính toán tương tự cho các bit còn lại (0 1 0 1 0) ta thu được chuỗi 16 bit lý tưởng sau khi mã hóa: 11 01 01 11 11 10 00 10

1.4.2 Quá trình Giải mã

Để đánh giá khả năng sửa lỗi của thuật toán Viterbi trong môi trường truyền dẫn có nhiễu, ta giả sử 2 cặp bit thứ 2 và thứ 3 bị sai lệch.

Chuỗi bit bộ giải mã thu được là: 11 11 00 11 11 10 00 10

Dựa trên Sơ đồ lưới Trellis (Hình 1.5), dưới đây là biểu diễn tính toán trong 8 chặng thời gian của bộ giải mã ($t_0 \rightarrow t_8$):

- Chặng 1 ($t_0 \rightarrow t_1$): Nhận cặp bit 11
 - Xuất phát từ trạng thái 00: ($PM_{00} = 0$). Sơ đồ lưới rẽ 2 nhánh:
 - Đến 00 (sinh 00) $\Rightarrow BM = 2 \Rightarrow$ Lỗi tích lũy $PM_{00} = 0 + 2 = 2$
 - Đến 10 (sinh 11) $\Rightarrow BM = 0 \Rightarrow$ Lỗi tích lũy $PM_{10} = 0 + 0 = 0$
- Chặng 2 ($t_1 \rightarrow t_2$): Nhận cặp bit 11
 - Chưa có hội tụ nhánh, khối ACS chỉ thực hiện Cộng (Add)
 - Từ 00 đến 00 (sinh 00) $\Rightarrow BM = 2 \Rightarrow PM_{00} = 2 + 2 = 4$
 - Từ 00 đến 10 (sinh 11) $\Rightarrow BM = 0 \Rightarrow PM_{10} = 2 + 0 = 2$
 - Từ 10 đến 01 (sinh 10) $\Rightarrow BM = 1 \Rightarrow PM_{01} = 0 + 1 = 1$
 - Từ 10 đến 11 (sinh 01) $\Rightarrow BM = 1 \Rightarrow PM_{11} = 0 + 1 = 1$
- Chặng 3 ($t_2 \rightarrow t_3$): Nhận cặp bit 00
 - Bắt đầu có hội tụ nhánh, khối ACS thực thi thêm hàm min (Compare & Select)
 - Đích 00: $\min(4 + 0, 1 + 2) \rightarrow$ Chọn nhánh từ 01. $PM_{00} = 3$
 - Đích 10: $\min(4 + 2, 1 + 0) \rightarrow$ Chọn nhánh từ 01. $PM_{10} = 1$
 - Đích 01: $\min(2 + 1, 1 + 1) \rightarrow$ Chọn nhánh từ 11. $PM_{01} = 2$
 - Đích 11: $\min(2 + 1, 1 + 1) \rightarrow$ Chọn nhánh từ 11. $PM_{11} = 2$

- Chặng 4 ($t_3 \rightarrow t_4$): Nhận cặp bit 11
 - Tiếp tục tính toán với các PM hiện tại
 - Đích 00: $\min(3 + 2, 2 + 0) \rightarrow$ Chọn nhánh từ 01. Cập nhật $PM_{00} = 2$
 - Đích 10: $\min(3 + 0, 2 + 2) \rightarrow$ Chọn nhánh từ 00. Cập nhật $PM_{10} = 3$
 - Đích 01: $\min(1 + 1, 2 + 1) \rightarrow$ Chọn nhánh từ 10. Cập nhật $PM_{01} = 2$
 - Đích 11: $\min(1 + 1, 2 + 1) \rightarrow$ Chọn nhánh từ 10. Cập nhật $PM_{11} = 2$
- Chặng 5 ($t_4 \rightarrow t_5$): Nhận cặp bit 11
 - Đích 00: $\min(2 + 2, 2 + 0) \rightarrow$ Chọn từ 01. $PM_{00} = 2$
 - Đích 10: $\min(2 + 0, 2 + 2) \rightarrow$ Chọn từ 00. $PM_{10} = 2$
 - Đích 01: $\min(3 + 1, 2 + 1) \rightarrow$ Chọn từ 11. $PM_{01} = 3$
 - Đích 11: $\min(3 + 1, 2 + 1) \rightarrow$ Chọn từ 11. $PM_{11} = 2$
- Chặng 6 ($t_5 \rightarrow t_6$): Nhận cặp bit 10
 - Đích 00: $\min(2 + 1, 3 + 1) \rightarrow$ Chọn từ 00. $PM_{00} = 3$
 - Đích 10: $\min(2 + 1, 3 + 1) \rightarrow$ Chọn từ 00. $PM_{10} = 3$
 - Đích 01: $\min(2 + 0, 3 + 2) \rightarrow$ Chọn từ 10. $PM_{01} = 2$
 - Đích 11: $\min(2 + 2, 3 + 0) \rightarrow$ Chọn từ 11. $PM_{11} = 3$
- Chặng 7 ($t_6 \rightarrow t_7$): Nhận cặp bit 00
 - Đích 00: $\min(3 + 0, 2 + 2) \rightarrow$ Chọn từ 00. $PM_{00} = 3$
 - Đích 10: $\min(3 + 2, 2 + 0) \rightarrow$ Chọn từ 01. $PM_{10} = 2$
 - Đích 01: $\min(3 + 1, 3 + 1) \rightarrow$ Chọn từ 10. $PM_{01} = 4$
 - Đích 11: $\min(3 + 1, 3 + 1) \rightarrow$ Chọn từ 10. $PM_{11} = 4$
- Chặng 8 ($t_7 \rightarrow t_8$): Nhận cặp bit 10
 - Đích 00: $\min(3 + 1, 4 + 1) \rightarrow$ Chọn từ 00. $PM_{00} = 4$
 - Đích 10: $\min(3 + 1, 4 + 1) \rightarrow$ Chọn từ 00. $PM_{10} = 4$
 - Đích 01: $\min(2 + 0, 4 + 2) \rightarrow$ Chọn từ 10. $PM_{01} = 2$
 - Đích 11: $\min(2 + 2, 4 + 0) \rightarrow$ Chọn từ 10. $PM_{11} = 4$

Sau khi tính toán xong thuật toán thực hiện Traceback toàn bộ chuỗi:

Đến cuối thời điểm t_8 , ta có 4 giá trị *Path Metric*:

- $PM_{00} = 4$
- $PM_{10} = 4$
- $PM_{01} = 2$
- $PM_{11} = 4$

=> Tổng lỗi tích lũy của nhánh nhỏ nhất $PM_{01} = 2$. Ta chọn trạng thái 01 ở t_8 để làm điểm xuất phát Traceback:

- Từ t_8 : Trạng thái 01 được ghi nhớ là đi đến từ 10 (tương ứng nhánh có Input = 0).
- Về t_7 : Trạng thái 10 được ghi nhớ là đi đến từ 01 (Input = 1)
- Về t_6 : Trạng thái 01 được ghi nhớ là đi đến từ 10 (Input = 0)
- Về t_5 : Trạng thái 10 được ghi nhớ là đi đến từ 00 (Input = 1)
- Về t_4 : Trạng thái 00 được ghi nhớ là đi đến từ 01 (Input = 0)
- Về t_3 : Trạng thái 01 được ghi nhớ là đi đến từ 11 (Input = 0)
- Về t_2 : Trạng thái 11 được ghi nhớ là đi đến từ 10 (Input = 1)
- Về t_1 : Trạng thái 10 được ghi nhớ là đi đến từ 00 (Input = 1)

Đảo ngược chuỗi Input thu được, ta được bản tin gốc 1 1 0 0 1 0 1 0. Hoàn toàn chính xác mặc dù ta đã giả định quá trình truyền tin bị lỗi

Trong quá trình giải mã thực tế với môi trường nhiễu phức tạp, hệ thống có thể gặp phải trường hợp tại chặng cuối cùng (hoặc ngay trong các bước ACS), có 2 hay nhiều trạng thái mang cùng một giá trị PM nhỏ nhất.

Khi triển khai trên phần cứng, mạch logic không có khả năng "chọn ngẫu nhiên". Do đó, thiết kế cần tích hợp cơ chế phân xử ưu tiên (Priority Tie-breaking) bên trong khối tìm Min. Giải pháp phổ biến là lập trình cho mạch luôn ưu tiên chọn trạng thái có chỉ số (index) nhỏ nhất làm điểm xuất phát cho quá trình Traceback. Ví dụ nếu $PM_{00} = PM_{01} = \min$, mạch sẽ tự động chọn nhánh 00

1.5 Ứng dụng của Thuật toán Viterbi

Nhờ khả năng tìm kiếm chuỗi trạng thái tối ưu và sửa lỗi vô cùng mạnh mẽ, thuật toán Viterbi được ứng dụng rộng rãi trong các hệ thống yêu cầu độ tin cậy dữ liệu cao, đặc biệt là khi tín hiệu phải đi qua các môi trường nhiễu nhiễu. Các ứng dụng tiêu biểu bao gồm:

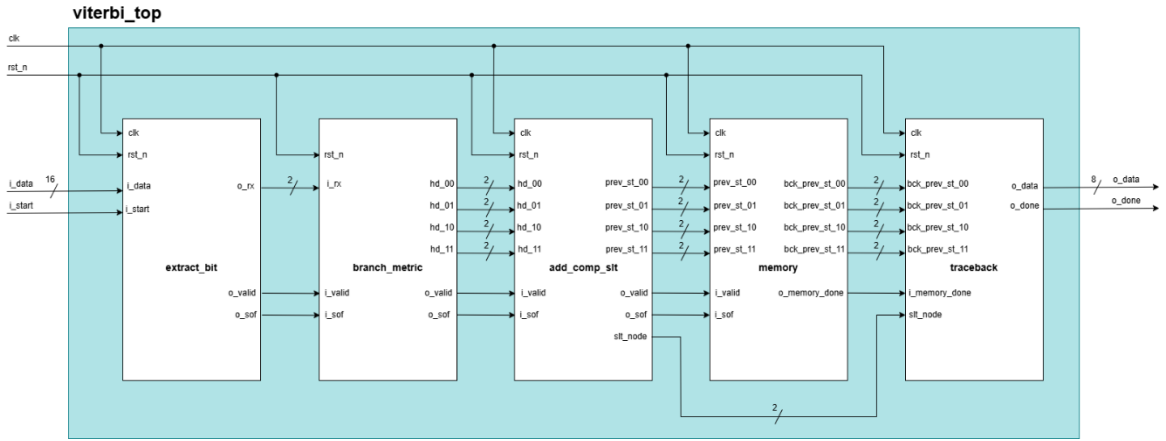
- **Hệ thống Viễn thông:** Đây là lĩnh vực ứng dụng kinh điển nhất của Viterbi. Thuật toán được tích hợp sâu trong các bộ thu phát để giải mã dữ liệu truyền qua không gian. Nó là chuẩn bắt buộc trong các mạng di động (GSM, 3G, mạng LTE), mạng không dây (Wi-Fi 802.11), truyền hình kỹ thuật số (DVB) và thông tin liên lạc vệ tinh
- **Thiết bị lưu trữ dữ liệu (Data Storage):** Trong các ổ đĩa cứng từ tính (HDD) hoặc đĩa quang (CD/DVD), khi đầu từ đọc dữ liệu ở tốc độ cao, tín hiệu analog thu được thường bị méo và nhiễu (hiện tượng can nhiễu liên ký tự - ISI). Viterbi được sử dụng trong vi điều khiển của ổ cứng (kết hợp với thuật toán PRML) để dự đoán và khôi phục lại chính xác chuỗi bit 0 và 1 từ các tín hiệu vật lý mờ hồ đó

CHƯƠNG 2. THIẾT KẾ BỘ GIẢI MÃ VITERBI

Chương 2 là phần trình bày chi tiết về đặc tả kỹ thuật (specification) của bộ giải mã Viterbi với hệ số mã chập $K = 3$, tốc độ $R = 1/2$. Đặc điểm chính của thiết kế này nằm ở việc sử dụng kiến trúc Pipelined Dataflow, áp dụng cơ chế “hand-shake” trực tiếp giữa các khối để có thể tối ưu hơn về tốc độ xử lý, giảm độ trễ và tiết kiệm tài nguyên phần cứng. Nội dung của chương sẽ lần lượt đi sâu vào phân tích sơ đồ khối, nguyên lý hoạt động và đặc tả giao tiếp của module cấp cao nhất (*viterbi_top*) và 5 module thành phần (*extract_bit*, *branch_metric*, *add_comp_slts*, *memory* và *traceback*).

2.1 Module *viterbi_top*

2.1.1 Sơ đồ khối



Hình 2.1: Block Diagram Module top

2.1.2 Chức năng

Module *viterbi_top* là module cấp cao nhất (top-level module) của toàn bộ thiết kế, đóng vai trò như một lớp vỏ tích hợp và quản lý luồng dữ liệu của bộ giải mã Viterbi.

Đặc điểm chính của kiến trúc này là việc áp dụng mô hình Pipelined Dataflow. Thay vì sử dụng khối điều khiển FSM trung tâm, 5 module thành phần bên trong (*extract_bit*, *branch_metric*, *add_comp_slts*, *memory* và *traceback*) được kết nối trực tiếp với nhau và tự động luân chuyển dữ liệu thông qua các tín hiệu “hand-shake”.

Chức năng chính của module *viterbi_top* là tiếp nhận một luồng dữ liệu mã hóa đầu vào có độ rộng 16-bit, tự động xử lý qua các khối và khôi phục thành công luồng dữ liệu gốc có độ rộng 8-bit dựa trên sơ đồ lưới Trellis với các tham số hệ số mã chập $K = 3$, tốc độ $R = 1/2$ đã được nêu ở Chương 1.

2.1.3 Hoạt động

Toàn bộ hệ thống bên trong *viterbi_top* hoạt động đồng bộ theo sườn lên của xung nhịp clk. Khi tín hiệu reset bất đồng bộ rst_n = 0 (Active-Low), tất cả các module thành phần bên trong đều được reset, xóa sạch các thanh ghi về giá trị khởi tạo. Quá trình giải mã chuỗi 16 bit bắt đầu khi tín hiệu i_start được cấp một xung ở mức cao kéo dài 1 chu kỳ.

- Luồng dữ liệu sẽ trải qua 2 pha chính:
 - **Bắt đầu giải mã (Forward Phase):** Khi quá trình giải mã được kích hoạt bởi tín hiệu i_start, chuỗi 16-bit đầu vào (i_data) sẽ được nạp vào khối extract_bit. Luồng dữ liệu sau đó tự động chảy qua các tầng: *extract_bit* (trích xuất cặp bit) → *branch_metric* (tính toán khoảng cách Hamming) → *add_comp_slr* (cộng/so sánh/chọn) → *memory* (ghi vào bộ nhớ).
 - **Truy vết (Traceback Phase):** Sau khi ghi đủ 8 cặp trạng thái, khối *memory* sẽ tự động chuyển sang pha đọc và phát cờ đánh thức khối *traceback* thực hiện việc dò ngược sơ đồ Trellis để tìm lại chuỗi gốc.
- Tổng thời gian từ lúc nhận cờ i_start và chuỗi bit vào i_data đến lúc giải mã xong được ước tính mất khoảng 19 chu kỳ clk (Latency = 19 clk). Khi quá trình hoàn tất, dữ liệu hợp lệ sẽ được xuất ra qua tín hiệu o_data, đồng thời tự động kéo cờ o_done lên mức 1 trong 1 chu kỳ để báo hiệu việc giải mã hoàn tất cho hệ thống bên ngoài. Nếu không có i_start mới, hệ thống tự động rơi vào trạng thái nghỉ để tiết kiệm năng lượng.

2.1.4 Mô tả tín hiệu vào/ra (I/O)

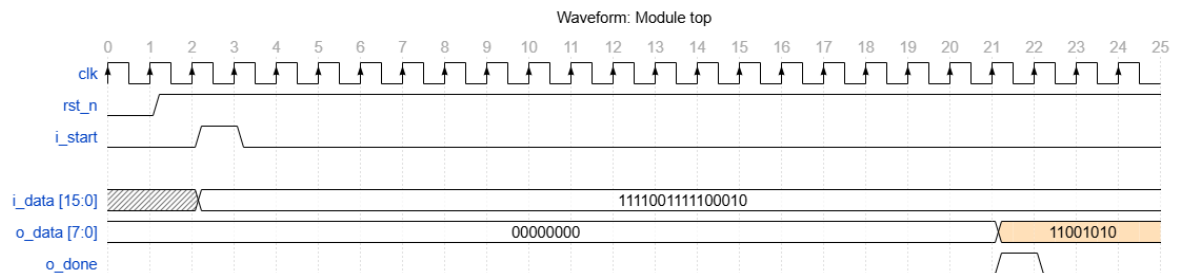
Tổng quan tín hiệu I/O:

Bảng 2.1: Mô tả tín hiệu I/O module top

Port	Width	Direction	Description
clk	1-bit	Input	Xung nhịp đồng hồ, điều khiển hoạt động đồng bộ của toàn hệ thống theo sườn lên
rst_n	1-bit	Input	Tín hiệu reset bất đồng bộ toàn hệ thống, tích cực mức thấp (Active Low)
i_start	1-bit	Input	Tín hiệu kích hoạt. Kéo lên mức 1 trong 1 chu kỳ để báo hiệu nạp dữ liệu và bắt đầu giải mã

i_data	16-bit	Input	Chuỗi dữ liệu mã hóa đầu vào cần giải mã
o_data	8-bit	Output	Dữ liệu 8-bit gốc đã được giải mã hoàn chỉnh
o_done	1-bit	Output	Cờ báo hiệu hoàn tất. Kéo lên mức 1 trong 1 chu kỳ khi o_data hợp lệ

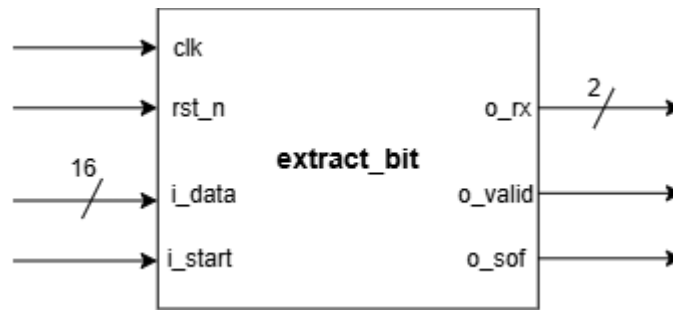
2.1.5 Waveform



Hình 2.2: Waveform module top

2.2 Module extract_bit

2.2.1 Sơ đồ khối



Hình 2.3: Block Diagram Module extract_bit

2.2.2 Chức năng

Module *extract_bit* đóng vai trò là cửa ngõ tiếp nhận dữ liệu của toàn bộ hệ thống giải mã.

Chức năng chính của module này là chuyển đổi luồng dữ liệu mã hóa 16-bit (*i_data*) thành một luồng dữ liệu nối tiếp gồm các cặp 2-bit (*o_rx*). Các cặp bit này được trích xuất tuần tự từ MSB tới LSB của chuỗi 16-bit để cung cấp dữ liệu liên tục cho khối tính toán khoảng cách Hamming (*branch_metric*).

Module *extract_bit* còn thực hiện chức năng khác là khởi tạo luồng dữ liệu bằng cách tự động sinh ra các tín hiệu “hand-shake” là *o_valid* (báo hiệu dữ liệu hợp lệ) và *o_sof* (báo hiệu cặp bit đầu tiên của chuỗi).

2.2.3 Hoạt động

Module *extract_bit* được thiết kế dựa trên cấu trúc của một thanh ghi dịch (Shift Register) kết hợp với một bộ đếm lùi (Counter), hoạt động đồng bộ theo sườn lên của *clk* với chu trình:

- **Trạng thái nghỉ:** Khi *rst_n* = 0 hoặc hệ thống đang rảnh, thanh ghi dịch bị xóa, các output *o_rx*, *o_valid*, *o_sof* đều được giữ ở mức 0.
- **Nạp dữ liệu:** khi bắt gặp tín hiệu *i_start* = 1, module lập tức nạp toàn bộ 16 bit từ *i_data* vào thanh ghi dịch nội bộ. Đồng thời, bộ đếm được thiết lập để chuẩn bị đếm 8 chu kỳ
- **Chu kỳ đầu tiên:** Ngay sau khi nạp 16 bit vào thanh ghi dịch, 2 bit MSB của thanh ghi sẽ được xuất thẳng ra ngõ ra *o_rx*. Còn *o_valid* được kéo lên 1 để báo cho khối sau biết dữ liệu đã sẵn sàng, còn *o_sof* cũng được kéo lên để khối ACS và *memory* biết đây là điểm bắt đầu của một chuỗi bit mới (để reset lại các bộ tích lũy bên trong chúng).
- **Quá trình dịch bit:** Ở 7 chu kỳ xung nhịp tiếp theo, còn *o_sof* tự động được kéo xuống 0, còn *o_valid* vẫn giữ ở mức 1. Thanh ghi thực hiện

phép dịch trái 2 bit ở mỗi sườn clk, lần lượt đẩy các cặp bit tiếp theo lên vị trí MSB và xuất ra o_rx.

- **Kết thúc:** Sau khi đẩy đủ 8 cặp bit (đếm hết 8 nhịp), cờ o_valid sẽ được kéo về 0, module quay về trạng thái nghỉ, chờ xung kích hoạt i_start của chuỗi bit tiếp theo

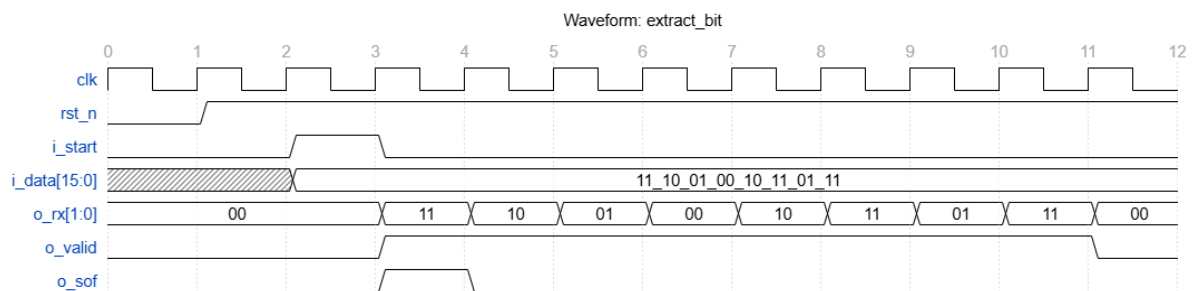
2.2.4 Mô tả tín hiệu vào/ra (I/O)

Tổng quan tín hiệu I/O:

Bảng 2.2: Mô tả tín hiệu I/O module *extract_bit*

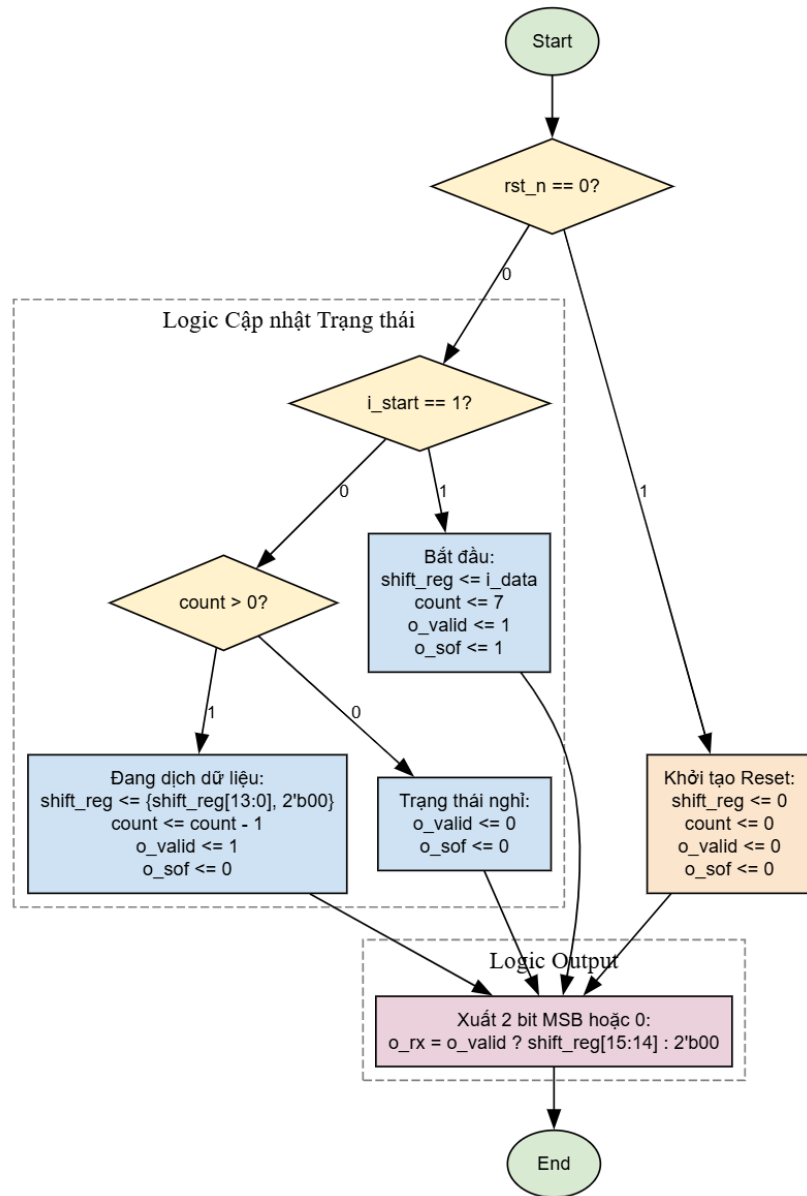
Port	Width	Direction	Description
clk	1-bit	Input	Xung nhịp đồng hồ, điều khiển hoạt động đồng bộ của mạch theo sườn lên
rst_n	1-bit	Input	Tín hiệu reset, đặt lại trạng thái thanh ghi dịch và các cờ báo hiệu về 0
i_start	1-bit	Input	Tín hiệu kích hoạt nạp dữ liệu. Báo hiệu có chuỗi 16-bit mới cần xử lý
i_data	16-bit	Input	Chuỗi dữ liệu mã hóa đầu vào rộng 16-bit
o_rx	2-bit	Output	Cặp bit liên tiếp được trích xuất ra để gửi tới khối <i>branch_metric</i>
o_valid	1-bit	Output	Cờ báo tín hiệu hợp lệ. Giữ mức 1 liên tục trong 8 chu kỳ khi o_rx đang xuất dữ liệu hợp lệ
o_sof	1-bit	Output	Cờ báo chuỗi bit đầu. Chỉ bật lên mức 1 ở chu kỳ đầu tiên của luồng 8 nhịp để đánh dấu cặp bit khởi đầu

2.2.5 Waveform



Hình 2.4: Waveform module *extract_bit*

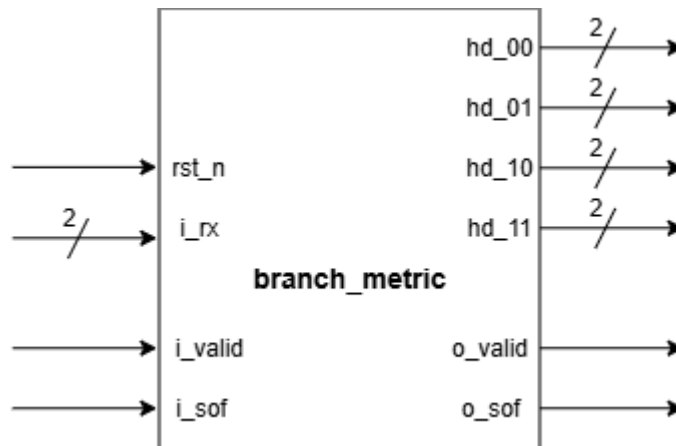
2.2.6 Lưu đồ thuật toán



Hình 2.5: Flowchart module extarct_bit

2.3 Module branch_metric

2.3.1 Sơ đồ khối



Hình 2.6: Block Diagram Module branch_metric

2.3.2 Chức năng

Module *branch_metric* đóng vai trò là bộ tiền xử lý số học của hệ thống giải mã Viterbi. Chức năng cốt lõi của nó là tính toán khoảng cách Hamming giữa cặp bit nhận được *i_rx* so với 4 giá trị nhánh chuẩn của sơ đồ lưới Trellis được nêu ở phần lý thuyết Chương 1.

Trong luồng Dataflow, module *branch_metric* nhận cờ *i_valid* và *i_sof* từ module *extract_bit* và đẩy thẳng sang ngõ ra để đồng bộ hóa hoàn hảo với dữ liệu toán học vừa được tính toán xong, tạo tiền đề cho module ACS phía sau hoạt động.

2.3.3 Hoạt động

Module *branch_metric* được thiết kế là 1 mạch tổ hợp (Combinational Logic), nó không cần sử dụng xung nhịp clk mà phản ứng ngay lập tức với sự thay đổi của ngõ vào:

- Tính toán khoảng cách Hamming: Khi nhận được một cặp bit *i_rx* từ module *extract_bit* gửi sang, module *branch_metric* sử dụng các cổng logic XOR và bộ cộng để so sánh từng bit của *i_rx* với 4 hằng số tham chiếu:
 - *hd_00*: Khoảng cách Hamming giữa *i_rx* và 2'b00
 - *hd_01*: Khoảng cách Hamming giữa *i_rx* và 2'b01
 - *hd_10*: Khoảng cách Hamming giữa *i_rx* và 2'b10
 - *hd_11*: Khoảng cách Hamming giữa *i_rx* và 2'b11
- Vì mỗi cặp có 2 bit, nên giá trị khoảng cách Hamming lớn nhất là 2. Do đó, độ rộng của các ngõ ra *hd_XX* được thiết kế tối ưu là 2-bit.

- Đồng bộ luồng dữ liệu: Các cờ “hand-shake” o_valid và o_sof đơn thuần là được gán bằng chính các tín hiệu i_valid và i_sof đầu vào, đảm bảo 2 cờ này được truyền sang module tiếp theo mà không bị lệch pha.
- Reset: Mặc dù là mạch tổ hợp, tín hiệu rst_n vẫn được sử dụng như một lớp bảo vệ. Khi rst_n = 0, toàn bộ các ngõ ra và cờ báo hiệu đều bị ép về giá trị 0 để tránh nhiễu rác lan truyền.

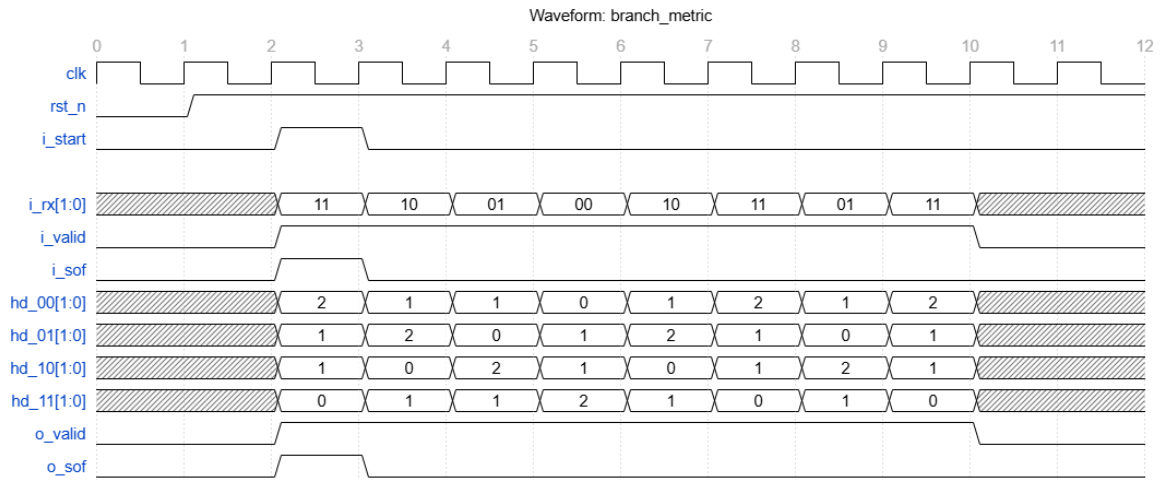
2.3.4 Mô tả tín hiệu vào/ra (I/O)

Tổng quan tín hiệu I/O:

Bảng 2.3: Mô tả tín hiệu I/O module branch_metric

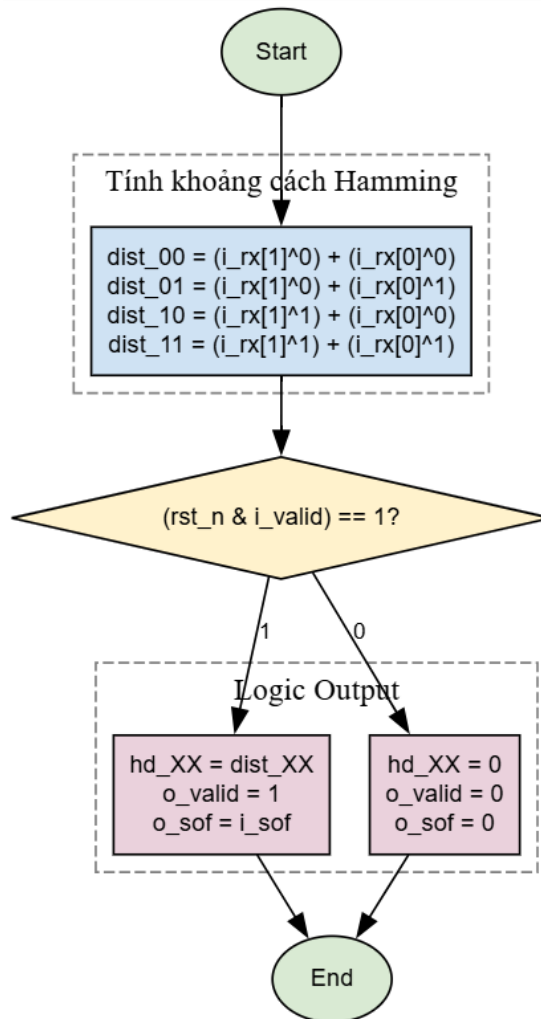
Port	Width	Direction	Description
rst_n	1-bit	Input	Tín hiệu reset, ép toàn bộ ngõ ra về 0 khi tích cực mức thấp
i_rx	2-bit	Input	Cặp bit nhận được từ khối <i>extract_bit</i>
i_valid	1-bit	Input	Cờ báo hiệu dữ liệu i_rx đang hợp lệ
i_sof	1-bit	Input	Cờ báo hiệu cặp bit đầu tiên của chuỗi
hd_00	2-bit	Output	Khoảng cách Hamming giữa i_rx và 00
hd_01	2-bit	Output	Khoảng cách Hamming giữa i_rx và 01
hd_10	2-bit	Output	Khoảng cách Hamming giữa i_rx và 10
hd_11	2-bit	Output	Khoảng cách Hamming giữa i_rx và 11
o_valid	1-bit	Output	Cờ truyền tiếp tín hiệu i_valid sang module ACS
o_sof	1-bit	Output	Cờ truyền tiếp tín hiệu i_sof sang module ACS

2.3.5 Waveform



Hình 2.7: Waveform module branch_metric

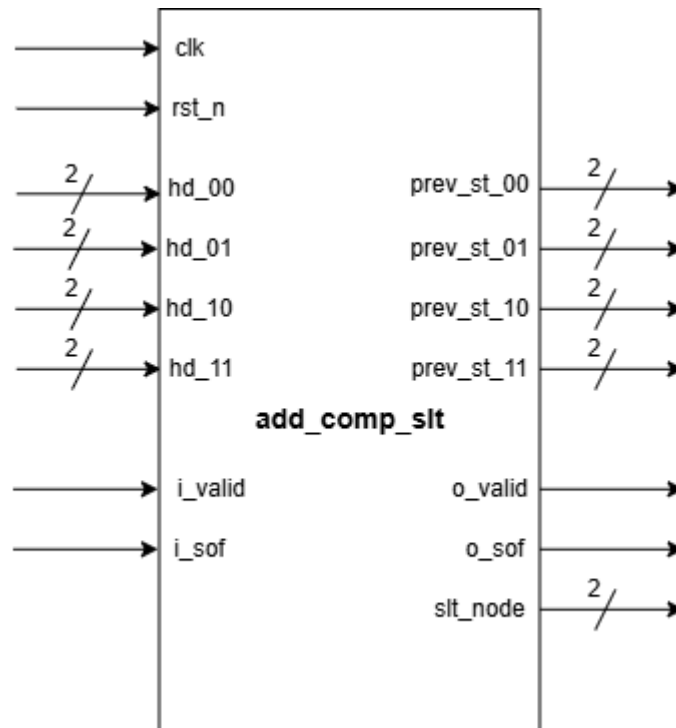
2.3.6 Lưu đồ thuật toán



Hình 2.8: Flowchart module branch_metric

2.4 Module add_comp_slt

2.4.1 Sơ đồ khối



Hình 2.9: Block Diagram Module ACS

2.4.2 Chức năng

Module *add_comp_slt* (ACS) có thể coi là “trái tim” của bộ giải mã Viterbi, nó là khối xử lý trung tâm, chịu trách nhiệm thực thi thuật toán Viterbi cốt lõi. Chức năng của khối là thực hiện đệ quy 3 thao tác liên tiếp: Cộng (Add) khoảng cách nhánh vào tổng tích lũy, So sánh (Compare) các con đường đi vào cùng một trạng thái, và Chọn (Select) con đường có tổng khoảng cách Path Metric nhỏ nhất như phần lý thuyết được trình bày ở Chương 1.

Kết quả của khối này là các survivor states được gửi tới bộ nhớ để phục vụ quá trình truy vết ngược, đồng thời xác định trạng thái có tổng sai số thấp nhất (*slt_node*) tại mỗi thời điểm.

2.4.3 Hoạt động

Module ACS hoạt động đồng bộ theo sườn lên của *clk*. Trong kiến trúc Dataflow, hoạt động của nó được điều khiển bởi cờ *i_sof* và *i_valid* để đảm bảo đồng bộ và chính xác tuyệt đối cho mỗi chuỗi dữ liệu. Khi cờ *i_valid* = 1, khối bắt đầu thực hiện tính toán và cập nhật Path Metric cho 4 trạng thái ở mỗi chu kỳ với cơ chế như sau:

- Cơ chế Khởi tạo: Đây là kỹ thuật cực kỳ quan trọng để ép bộ giải mã bắt đầu đúng từ trạng thái gốc của bộ mã hóa chấp. Khi cờ *i_sof* = 1 báo

hiệu cặp bit đầu tiên cần được xử lý (chu kỳ đầu tiên trên sơ đồ Trellis), module ACS sẽ tự động nạp giá trị khởi tạo cho 4 thanh ghi Path Metric nội bộ có độ rộng 5-bit (trường hợp xấu nhất theo lý thuyết, khoảng cách Hamming ở 8 chặng đều bằng 2 thì $PM_{max} = 8 \times 2 = 16$, sử dụng 5-bit chứa tối đa là 31 để tránh tràn số):

- pm_00 = 0: Trạng thái gốc có sai số bằng 0
- pm_01, pm_10, pm_11 = 23: Các trạng thái còn lại được gán 1 giá trị ảo rất lớn (lớn hơn giá trị PM_{max} theo lý thuyết)
- *Ý nghĩa:* Việc này tạo ra một mức ưu tiên cho trạng thái 00 ở chu kỳ 1, buộc mạch phải ưu tiên cho con đường xuất phát từ nút này theo đúng cơ sở lý thuyết của sơ đồ Trellis đã nêu ở Chương 1. Ở chu kỳ sau, dựa theo Logic Add–Compare–Select mà các giá trị ảo này sẽ bị loại bỏ hoàn toàn mà không làm ảnh hưởng tới tính chính xác của thuật toán.
- **Logic Add–Compare–Select (ACS):** Dựa trên cấu trúc của mã chập $K = 3, R = 1/2$, 8 nhánh của lưới Trellis được ánh xạ tối ưu vào 4 giá trị khoảng cách Hamming cơ sở. Các công thức tính tổng tích lũy mới (pmXX_next) và chọn trạng thái trước đó (prev_st_XX) được thực hiện như sau:
 - Trạng thái 00 nhận đường đi từ 00 (mẫu 00) hoặc 01 (mẫu 11):

$$pm00_{next} = \min (pm00 + hd_{00}, pm01 + hd_{11}) \quad PT\ 2-1$$
 - Trạng thái 10 nhận đường đi từ 00 (mẫu 11) hoặc 01 (mẫu 00):

$$pm10_{next} = \min (pm00 + hd_{11}, pm01 + hd_{00}) \quad PT\ 2-2$$
 - Trạng thái 01 nhận đường đi từ 10 (mẫu 10) hoặc 11 (mẫu 01):

$$pm01_{next} = \min (pm10 + hd_{10}, pm11 + hd_{01}) \quad PT\ 2-3$$
 - Trạng thái 11 nhận đường đi từ 10 (mẫu 01) hoặc 11 (mẫu 10):

$$pm11_{next} = \min (pm10 + hd_{01}, pm11 + hd_{10}) \quad PT\ 2-4$$
- Mạch so sánh sẽ chọn giá trị min để cập nhật vào pmXX cho chu kỳ tiếp theo, đồng thời ghi nhận lại trạng thái nguồn tạo ra giá trị min đó để xuất ra các tín hiệu prev_st_00, prev_st_01, prev_st_10, prev_st_11 gửi sang khối memory.

- Song song với quá trình trên, một khối logic tổ hợp sẽ liên tục so sánh 4 tổng pmXX hiện tại để tìm ra giá trị nhỏ nhất. Nút chứa giá trị nhỏ nhất này sẽ được xuất ra qua tín hiệu `slt_node` đưa tới khối traceback để phục vụ quá trình truy vết. Trong trường hợp các tổng bằng nhau, mạch sẽ áp dụng theo thứ tự ưu tiên cố định $00 > 10 > 01 > 11$

Khi tín hiệu `rst_n = 0`, tất cả các thanh ghi PM nội bộ, các ngõ ra đều bị xóa hoàn toàn về giá trị 0, việc này đảm bảo mạch không ở trạng thái bất định khi hệ thống vừa khởi động.

Khi cờ `i_valid = 0`, tức là khối phía trước chưa sẵn sàng hoặc đang trong trạng thái nghỉ, khối ACS sẽ rơi vào trạng thái giữ giá trị. Tất cả các thanh ghi PM nội bộ và các ngõ ra `prev_st_XX`, `slt_node` sẽ giữ nguyên giá trị của các chu kỳ trước đó, không thực hiện các phép tính cộng dồn hay so sánh mới, 2 cờ “hand-shake” `o_valid` và `o_sof` gửi sang khối memory sẽ được kéo về 0.

2.4.4 Mô tả tín hiệu vào/ra (I/O)

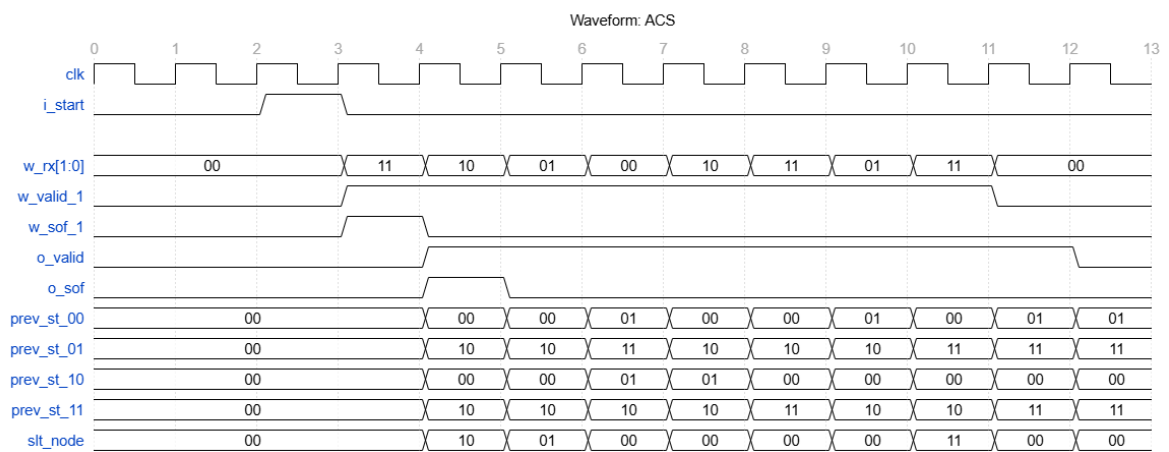
Tổng quan tín hiệu I/O:

Bảng 2.4: Mô tả tín hiệu I/O module ACS

Port	Width	Direction	Description
clk	1-bit	Input	Xung nhịp clock hệ thống
rst_n	1-bit	Input	Reset Active Low
i_valid	1-bit	Input	Cờ báo dữ liệu đầu vào hợp lệ
i_sof	1-bit	Input	Cờ báo cặp bit đầu tiên
hd_00	2-bit	Input	Khoảng cách Hamming giữa cặp bit đang xét và 00 từ khối <i>branch_metric</i>
hd_01	2-bit	Input	Khoảng cách Hamming của cặp bit đang xét so với 01 từ khối <i>branch_metric</i>
hd_10	2-bit	Input	Khoảng cách Hamming của cặp bit đang xét so với 10 từ khối <i>branch_metric</i>
hd_11	2-bit	Input	Khoảng cách Hamming của cặp bit đang xét so với 11 từ khối <i>branch_metric</i>
prev_st_00	2-bit	Output	Trạng thái liên trước được chọn của State 00 (Survivor State) gửi tới khối <i>memory</i>
prev_st_01	2-bit	Output	Trạng thái liên trước được chọn của State 01 (Survivor State) gửi tới khối <i>memory</i>

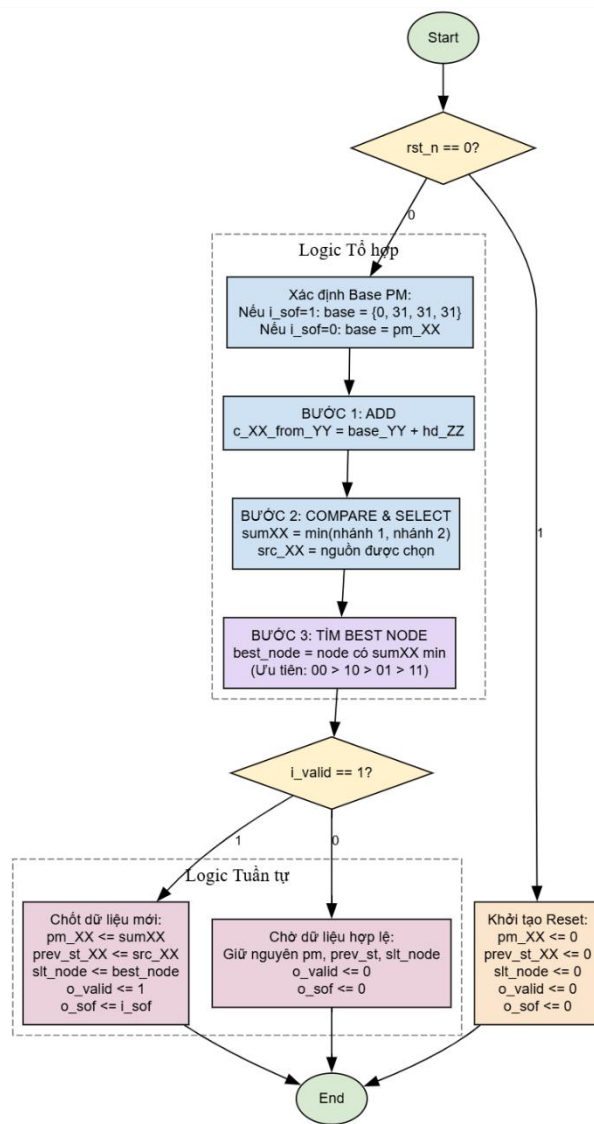
prev_st_10	2-bit	Output	Trạng thái liền trước được chọn của State 10 (Survivor State) gửi tới khối <i>memory</i>
prev_st_11	2-bit	Output	Trạng thái liền trước được chọn của State 11 (Survivor State) gửi tới khối <i>memory</i>
slt_node	2-bit	Output	Trạng thái có Path Metric nhỏ nhất hiện tại gửi tới khối <i>traceback</i>
o_valid	1-bit	Output	Cờ báo hiệu dữ liệu ACS xuất ra là hợp lệ
o_sof	1-bit	Output	Cờ sof đã được đồng bộ trễ 1 nhịp

2.4.5 Waveform



Hình 2.10: Waveform module ACS

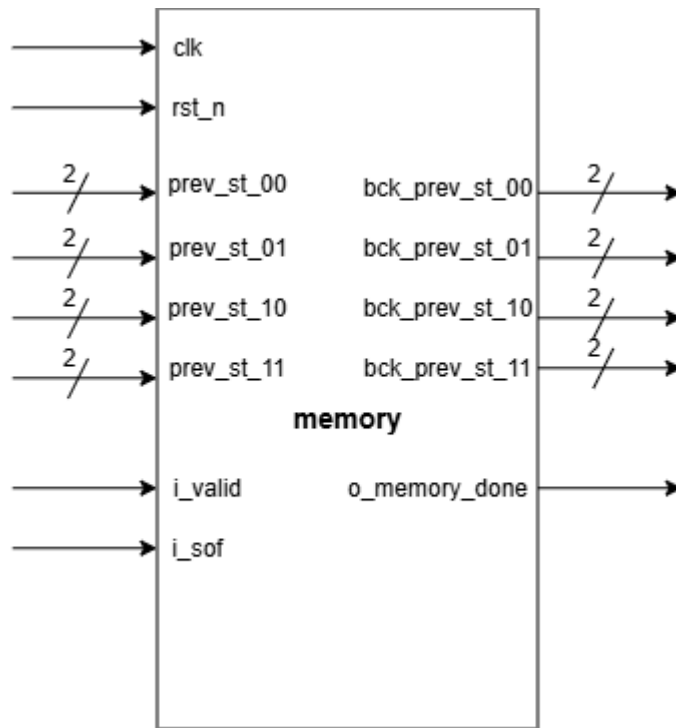
2.4.6 Lưu đồ thuật toán



Hình 2.11: Flowchart module ACS

2.5 Module memory

2.5.1 Sơ đồ khối



Hình 2.12: Block Diagram Module memory

2.5.2 Chức năng

Module *memory* hoạt động như một trạm trung chuyển kết nối giữa Forward Phase của khối ACS và Traceback Phase của khối *traceback*.

Nhiệm vụ cốt lõi của module này là ghi nhận và lưu trữ toàn bộ “dấu chân”, tức là 4 trạng thái liên trước *prev_st_00* tới *prev_st_11* do khối ACS quyết định trong suốt 8 chu kỳ. Sau khi ghi đủ 1 khung dữ liệu, module tự động chuyển trạng thái, lật ngược thời gian và xuất các dấu chân này theo thứ tự lùi từ chu kỳ 7 về chu kỳ 0 để cung cấp bản đồ đường đi cho khối *traceback* thực hiện truy vết.

2.5.3 Hoạt động

Module *memory* sử dụng 1 mảng 2 chiều [1:0] *trellis_diagr* [0:3][0:7] đóng vai trò bộ nhớ, trong đó 4 hàng tương ứng với 4 trạng thái, 8 cột tương ứng với 8 thời điểm, mỗi ô chứa 2 bit để lưu trạng thái *prev_st_XX* được khối ACS gửi sang. Khối *memory* sử dụng cờ nội bộ *read_mode* để kiểm soát quá trình Ghi/Đọc (0: Ghi, 1: Đọc), cùng với 2 bộ đếm *count*, *trace* phục vụ 2 quá trình này.

	0	1	...	7
0	2-bit	2-bit	...	2-bit
1	2-bit	2-bit	...	2-bit
2	2-bit	2-bit	...	2-bit
3	2-bit	2-bit	...	2-bit

Hình 2.13: Minh họa mảng nhớ module memory

Module *memory* hoạt động đồng bộ theo sườn lên clk, được chia thành 2 pha rõ rệt, quản lý bởi bộ đếm nội bộ:

- Khởi tạo và Reset: khi $rst_n = 0$, toàn bộ mảng nhớ, bộ đếm địa chỉ và các ngõ ra đều được ép về 0, cờ nội bộ $read_mode$ kéo về 0 chuẩn bị cho pha Ghi
- **Pha Ghi – Forward Phase:**
 - Khi cờ báo dữ liệu hợp lệ $i_valid = 1$, cờ báo cặp bit đầu $i_sof = 1$, cờ $read_mode = 0$. Module reset bộ đếm tiến (count) về 0 rồi thực hiện đếm từ 0 tới 7.
 - Ở mỗi chu kỳ xung nhịp, 4 giá trị $prev_st_00$ tới $prev_st_11$ từ khối ACS gửi sang sẽ được ghi vào một cột ở địa chỉ ‘count’ hiện tại của mảng nhớ (tương ứng 8 chu kì ghi lần lượt từ trái sang phải, từ cột 0 tới cột 7) lần lượt vào hàng 0 tới 3
- Chuyển pha và báo hiệu:
 - Ngay khi bộ đếm tiến count đạt đến giá trị 7 (đã ghi đủ 8 cột dấu chân), khung dữ liệu đã hoàn tất.
 - Tại sườn xung nhịp clk tiếp theo, module kéo cờ $o_memory_done = 1$ trong đúng 1 chu kì. Đây chính là tín hiệu “hand-shake” gửi tới module *traceback* đánh thức module này bắt đầu quá trình truy vết. Đồng thời mạch reset bộ đếm tiến count về 0 để chuẩn bị cho chuỗi bit mới, kéo cờ $read_mode$ lên 1 để chuyển sang Pha Đọc, bộ đếm lùi trace cũng được set bằng 7 để chuyển bị đếm ngược từ 7 về 0 phục vụ cho Pha Đọc
- **Pha Đọc – Traceback Phase:**
 - Khi mạch chuyển sang Pha Đọc, bộ đếm lùi được kích hoạt đếm ngược từ 7 về 0. Ở mỗi nhịp, module trích xuất 1 cột nhớ ở địa chỉ ‘trace’ hiện tại của bộ đếm (tương ứng với 8 chu kì lần lượt đọc từ phải sang trái, từ cột 7 về cột 0) và đưa ra 4 ngõ ra

bck_prev_st_XX gửi tới khối *traceback* phục vụ quá trình truy vết.

- Để đảm bảo sự ổn định và chống nhiễu cho module *traceback*, khi cờ read_mode đang ở mức 0, 4 ngõ ra bck_prev_st_XX đều bị ép về 0, chúng chỉ được phép xuất dữ liệu thật sự khi module *memory* đang ở Pha Đọc.

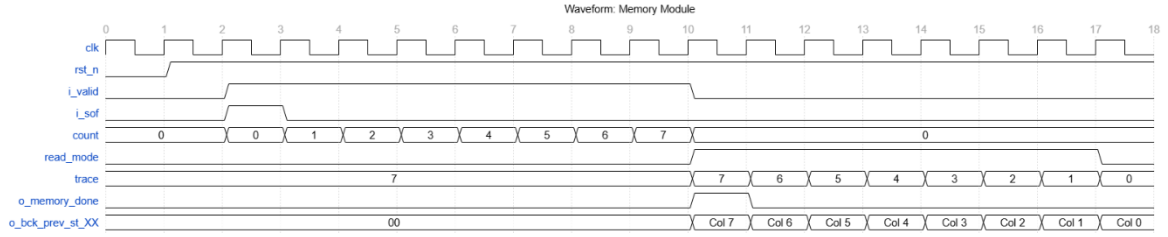
2.5.4 Mô tả tín hiệu vào/ra (I/O)

Tổng quan tín hiệu I/O:

Bảng 2.5: Mô tả tín hiệu I/O module *memory*

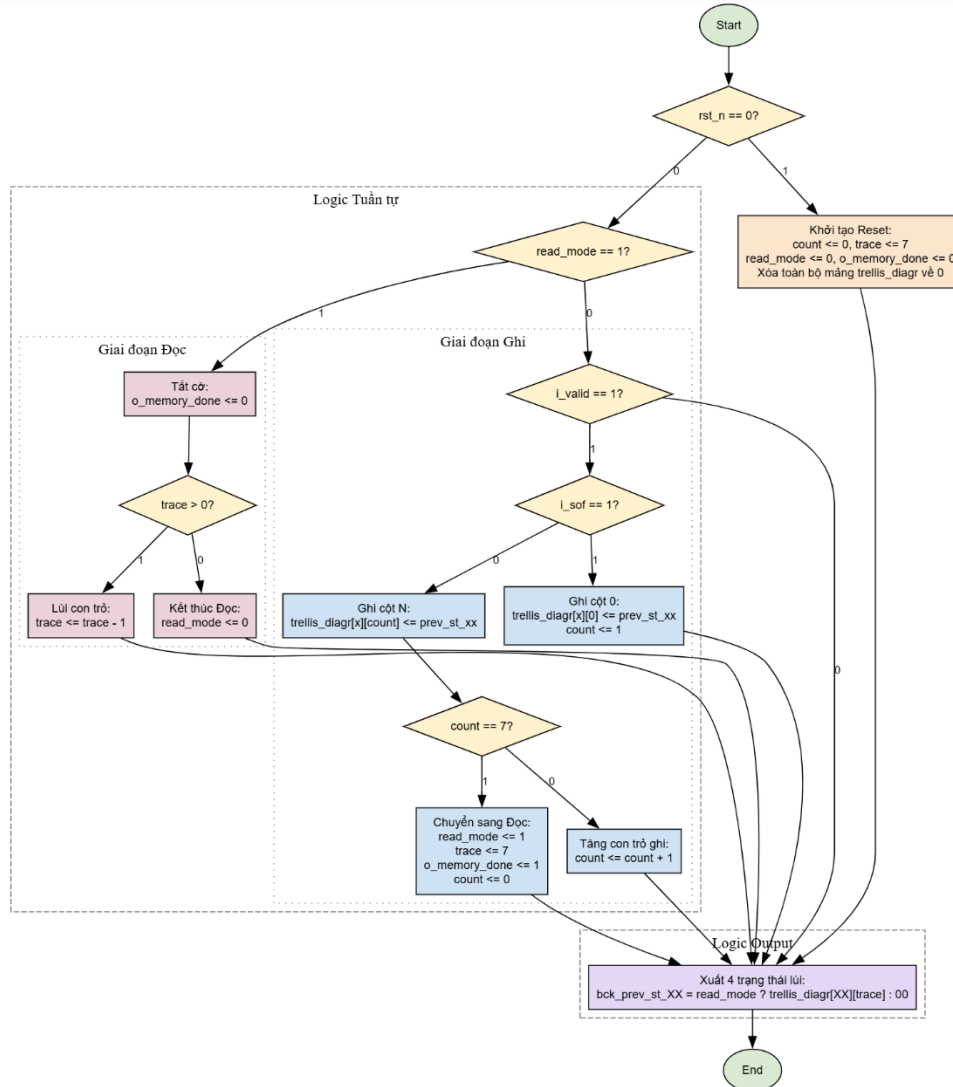
Port	Width	Direction	Description
clk	1-bit	Input	Xung nhịp Clock hệ thống
rst_n	1-bit	Input	Reset Active Low
i_valid	1-bit	Input	Cờ báo dữ liệu đầu vào hợp lệ
i_sof	1-bit	Input	Cờ báo hiệu bắt đầu khung từ ACS, dùng để ép bộ đếm ghi về 0
prev_st_00	2-bit	Input	Giá trị trạng thái liên trước của trạng thái 00 do khối ACS cung cấp
prev_st_01	2-bit	Input	Giá trị trạng thái liên trước của trạng thái 01 do khối ACS cung cấp
prev_st_10	2-bit	Input	Giá trị trạng thái liên trước của trạng thái 10 do khối ACS cung cấp
prev_st_11	2-bit	Input	Giá trị trạng thái liên trước của trạng thái 11 do khối ACS cung cấp
bck_prev_st_00	2-bit	Output	Giá trị trạng thái liên trước của trạng thái 00 được xuất lùi từ bộ nhớ ra
bck_prev_st_01	2-bit	Output	Giá trị trạng thái liên trước của trạng thái 01 được xuất lùi từ bộ nhớ ra
bck_prev_st_10	2-bit	Output	Giá trị trạng thái liên trước của trạng thái 10 được xuất lùi từ bộ nhớ ra
bck_prev_st_11	2-bit	Output	Giá trị trạng thái liên trước của trạng thái 11 được xuất lùi từ bộ nhớ ra
o_memory_done	1-bit	Output	Cờ báo hiệu quá trình Ghi đã hoàn tất. Nảy mức 1 trong 1 chu kỳ để đánh thức module <i>traceback</i>

2.5.5 Waveform



Hình 2.14: Waveform module memory

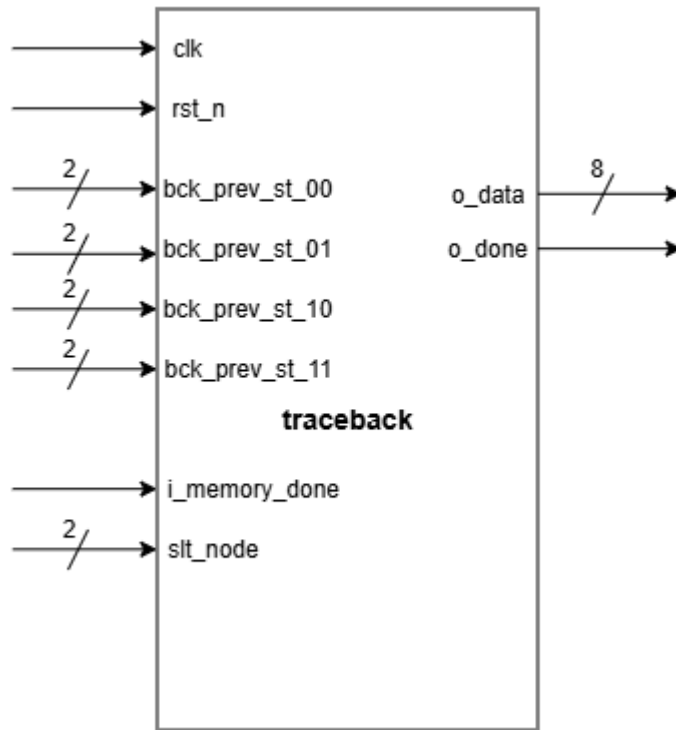
2.5.6 Lưu đồ thuật toán



Hình 2.15: Flowchart module memory

2.6 Module traceback

2.6.1 Sơ đồ khối



Hình 2.16: Block Diagram Module traceback

2.6.2 Chức năng

Module *traceback* đóng vai trò là thám tử giải mã ở khâu cuối cùng của hệ thống Viterbi. Chức năng chính của module này là thực hiện truy vết ngược dọc theo Survivor Path đã được ghi lại trong bộ nhớ để giải mã ra chuỗi 8-bit dữ liệu gốc trước khi mã hóa. Sau khi quá trình truy vết hoàn tất, nó xuất chuỗi bit giải mã được ra ngõ ra *o_data*, đồng thời kéo cờ báo hiệu giải mã hoàn tất *o_done* lên 1 chu kì.

2.6.3 Hoạt động

Module *traceback* ứng dụng một đặc điểm toán học của mã hóa chập để giải mã. Dựa trên đặc tính của bộ mã hóa chập $K = 3$ ở phía phát (bit dữ liệu mới luôn được dịch vào vị trí cao nhất của thanh ghi trạng thái), ta có quy luật bất di bất dịch: Trạng thái sinh ra có dạng 1_X (10, 11) chắc chắn bắt nguồn từ bit gốc là 1; và trạng thái 0_X (00, 01) chắc chắn bắt nguồn từ bit gốc là 0. Do đó, thay vì sử dụng FSM công kênh, khối *traceback* đơn giản chỉ cần sử dụng 1 dây dẫn lấy bit MSB của trạng thái hiện tại làm bit giải mã, giúp tiết kiệm tối đa tài nguyên.

Module *traceback* hoạt động đồng bộ theo sườn lên xung *clk*, khi *rst_n* = 0, các ngõ ra cùng các tín hiệu nội bộ được ép về 0. Hoạt động của nó diễn ra theo tuần tự 3 bước, được kích hoạt bởi cờ “hand-shake” *i_memory_done* từ khối *memory* gửi sang.

- **Kích hoạt và bắt đầu truy vết:** module *traceback* luôn ở trạng thái nghỉ cho tới khi cờ *i_memory_done* được kéo lên 1.
 - Ngay lập tức, mạch chốt giá trị trạng thái được chọn (*slt_node*) do khối ACS tính toán ra ở chu kỳ cuối pha Forward làm điểm bắt đầu quá trình traceback (gọi là *current_node*). Đồng thời, bộ đếm 8 nhịp count được kích hoạt.
- **Vòng lặp đi lùi và giải mã:** Trong 8 chu kỳ xung nhịp tiếp theo, mạch thực hiện lặp lại 2 thao tác song song:
 - *Trích xuất bit giải mã:* dựa theo quy tắc toán học về bit MSB vừa nêu ở trên, mạch chỉ cần trích xuất trực tiếp bit MSB của *current_node* hiện tại và đẩy nó vào 1 thanh ghi dịch chứa kết quả giải mã. Vì quá trình truy vết là đi lùi sơ đồ lưới Trellis, bit trích xuất được sẽ dịch dần từ MSB sang LSB của thanh ghi để ghép thành chuỗi bit cần giải mã hoàn chỉnh
 - *Lùi bước:* mạch sử dụng giá trị *current_node* hiện tại để làm chân selector cho 1 bộ MUX 4-1, nó sẽ chọn 1 trong 4 giá trị *i_bck_prev_st_XX* do khối *memory* gửi tới để cập nhật giá trị *current_node* mới cho chu kỳ tiếp theo.
- **Hoàn tất và xuất kết quả:** Khi bộ đếm count chạm mốc 8 (hoàn tất 8 bước lùi), chuỗi 8 bit gốc đã nằm trong thanh ghi dịch.
 - Tại nhịp clk tiếp theo, mạch gửi dữ liệu này vào ngõ ra *o_data*, đồng thời kéo cờ *o_done* lên 1 trong 1 chu kỳ để báo đã hoàn tất quá trình giải mã, bộ đếm count cũng được reset về 0 để chuẩn bị cho chuỗi bit tiếp theo.

2.6.4 Mô tả tín hiệu vào/ra (I/O)

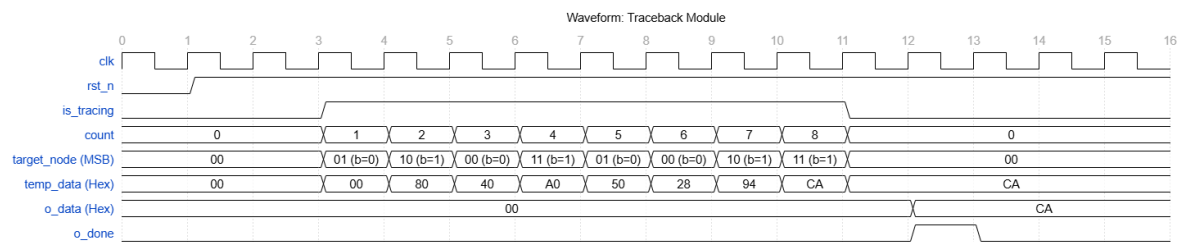
Tổng quan tín hiệu I/O:

Bảng 2.6: Mô tả tín hiệu I/O module *traceback*

Port	Width	Direction	Description
clk	1-bit	Input	Xung nhịp Clock hệ thống
rst_n	1-bit	Input	Reset Active Low
i_memory_done	1-bit	Input	Cờ “hand-shake” từ khối <i>memory</i> , kích hoạt quá trình Traceback
slt_node	2-bit	Input	Trạng thái có sai số thấp nhất tại cột cuối cùng
bck_prev_st_00	2-bit	Input	Giá trị dấu chân trạng thái liền trước của state 00 do khối <i>memory</i> gửi tới
bck_prev_st_01	2-bit	Input	Giá trị dấu chân trạng thái liền trước của state 01 do khối <i>memory</i> gửi tới

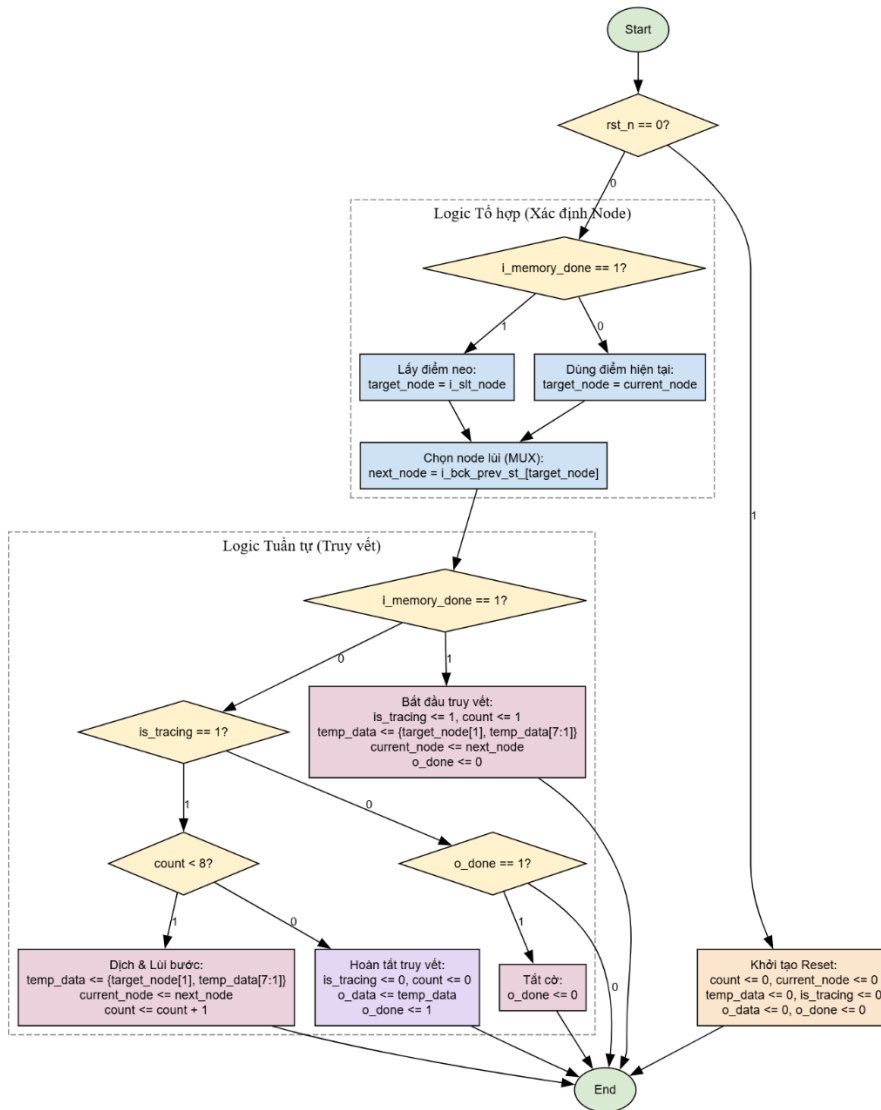
bck_prev_st_10	2-bit	Input	Giá trị dấu chân trạng thái liền trước của state 10 do khối <i>memory</i> gửi tới
bck_prev_st_11	2-bit	Input	Giá trị dấu chân trạng thái liền trước của state 11 do khối <i>memory</i> gửi tới
o_data	8-bit	Output	Chuỗi dữ liệu gốc đã được giải mã hoàn thiện
o_done	1-bit	Output	Cờ báo hiệu quá trình giải mã hoàn tất

2.6.5 Waveform



Hình 2.17: Waveform module traceback

2.6.6 Lưu đồ thuật toán



Hình 2.18: Flowchart module traceback

CHƯƠNG 3. KIỂM CHỨNG CHỨC NĂNG VÀ ĐÁNH GIÁ

Chương 3 sẽ trình bày chi tiết quá trình kiểm chứng chức năng (Functional Verification) cho bộ giải mã Viterbi đã được thiết kế. Quá trình kiểm thử được thực hiện theo phương pháp từ dưới lên, bắt đầu từ việc mô phỏng và đánh giá từng module nhỏ (sanity test) được thực hiện cùng lúc với quá trình thiết kế RTL để đảm bảo các module trong hệ thống hoạt động và phối hợp với nhau chính xác theo đặc tả ở Chương 2. Tiếp theo là đánh giá độ chính xác của toàn hệ thống dựa trên Golden Model được giảng viên cung cấp. Toàn bộ các kịch bản kiểm thử này đều được thực hiện trên môi trường QuestaSim. Cuối cùng, chương này đi sâu vào phân tích các chỉ số Code Coverage để đánh giá tính toàn vẹn của thiết kế RTL, chứng minh thiết kế hoàn toàn đáp ứng được với các yêu cầu đặc tả và sẵn sàng cho giai đoạn tổng hợp phần cứng.

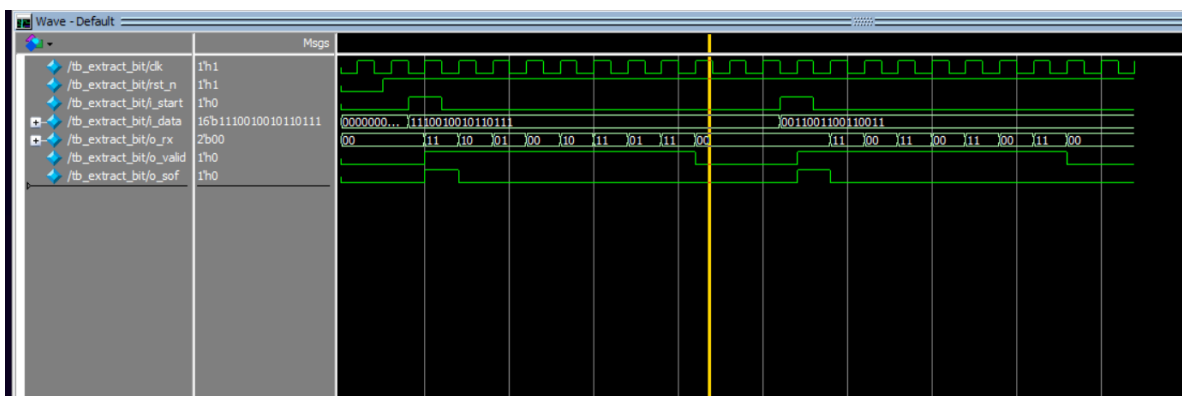
3.1 Kiểm chứng ở mức Module (Sanity test)

Mục đích: Giai đoạn Sanity test là giai đoạn đánh giá độc lập tính đúng đắn về mặt logic và timing của từng module riêng biệt trước khi tiến hành tích hợp chúng vào hệ thống tổng thể. Việc này được thực hiện song song với quá trình thiết kế RTL để đảm bảo code RTL của từng module không bị lỗi, hoạt động và timing của từng module được kiểm soát chặt chẽ theo phần đặc tả ở Chương 2 để chúng có thể phối hợp tốt với nhau.

Phương pháp: Giai đoạn Sanity test được thực hiện bằng cách viết các Testbench cơ bản, tạo ra các tín hiệu stimulus giả lập ngõ vào và quan sát sự thay đổi của ngõ ra thông qua waveform bằng công cụ QuestaSim.

3.1.1 Module extract_bit

Mục tiêu: Đối chiếu hoạt động và timing của khối extract_bit với phần đặc tả.



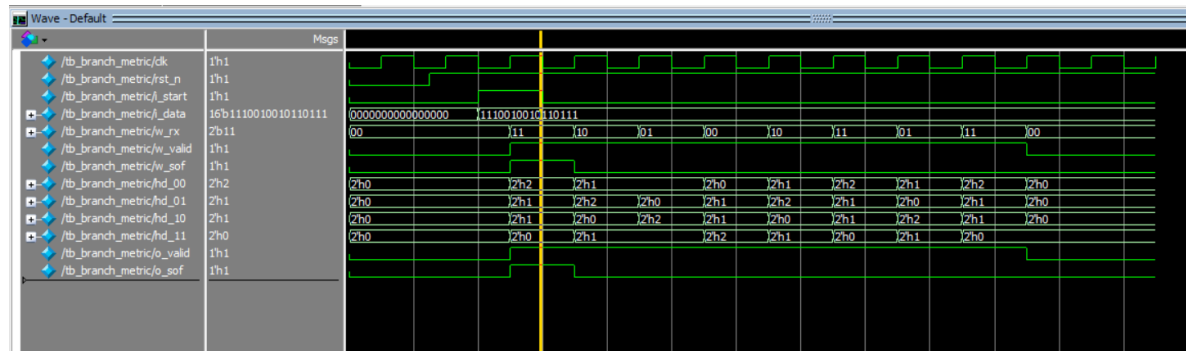
Hình 3.1: Waveform mô phỏng module extract_bit

Đánh giá: Khi nhận được xung kích hoạt i_start và dữ liệu vào 16-bit i_data, module bắt đầu tuần tự xuất cặp 2-bit ra ngõ ra o_rx ở mỗi sườn lên xung clk, 8 cặp bit được xuất ra không bị sai lệch so với 16-bit gốc, hoàn thành chính xác trong 8

chu kì. Cờ o_valid được giữ ở mức cao liên tục trong 8 chu kì, cờ o_sof chỉ lên mức 1 trong chu kì đầu tiên của khung truyền. Hết 8 chu kì lập tức cờ o_valid và ngõ ra o_rx được kéo về 0 để tối ưu năng lượng và tránh xuất những giá trị rác làm ảnh hưởng tới khả năng tính toán của các khối sau. Hoạt động và timing của khối hoàn toàn chính xác theo đặc tả.

3.1.2 Module branch_metric

Mục tiêu: Đối chiếu hoạt động và timing của khối branch_metric với phần đặc tả.

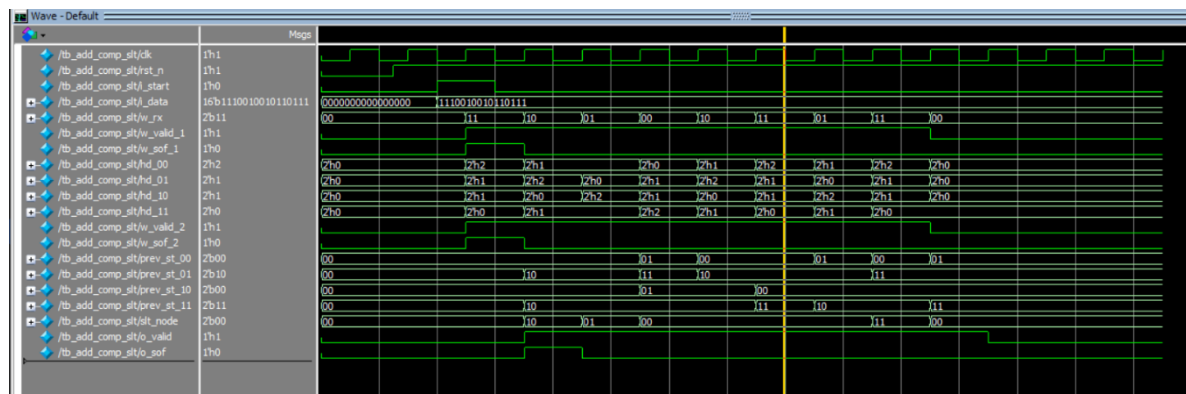


Hình 3.2: Waveform mô phỏng module branch_metric

Đánh giá: Waveform cho thấy các tín hiệu ngõ ra hd_00 đến hd_11 thay đổi giá trị ngay khi tín hiệu vào i_rx thay đổi mà không bị trễ qua sườn clk. Các giá trị khoảng cách Hamming được tính toán chính xác theo lý thuyết. 2 cờ o_valid và o_sof được truyền thành công qua module. Khi tín hiệu i_valid kéo về 0 lập tức các ngõ ra hd_00 tới hd_11 được ép về 0 để tối ưu năng lượng và tránh xuất các giá trị rác sang các khối sau. Hoạt động và timing của khối hoàn toàn chính xác theo đặc tả.

3.1.3 Module add_comp_slt

Mục tiêu: Đối chiếu hoạt động và timing của khối add_comp_slt với phần đặc tả.

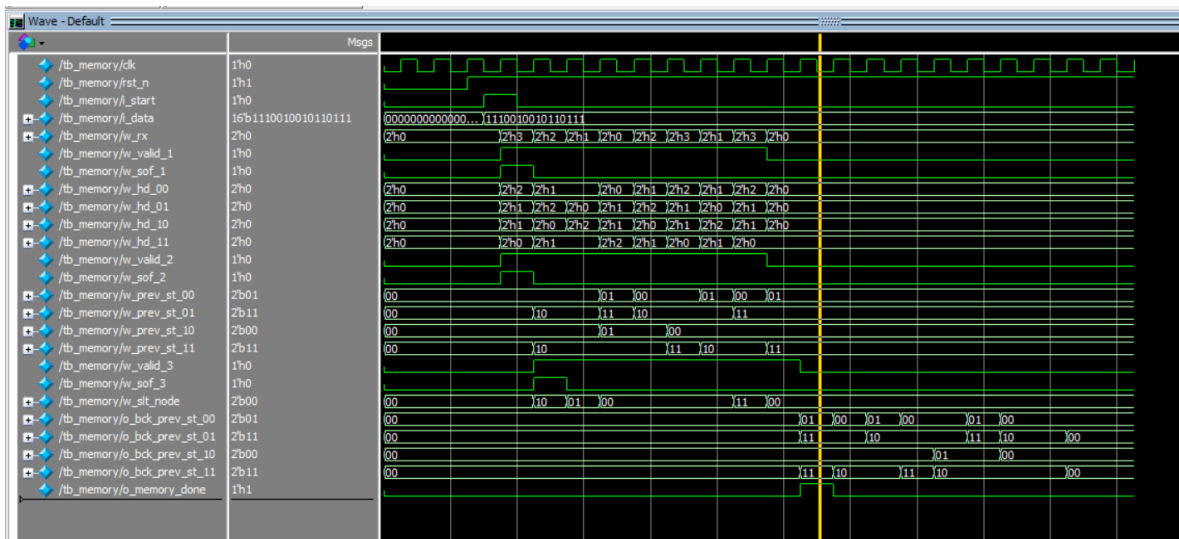


Hình 3.3: Waveform mô phỏng module add_comp_slt

Đánh giá: Waveform cho thấy module đã thực hiện các bước Add – Compare – Select và tính toán chính xác các giá trị trạng thái liên trước của 4 trạng thái prev_st_00 tới prev_st_11 dựa theo các giá trị khoảng cách Hamming đầu vào, đồng thời tính toán chính xác giá trị node tối ưu slt_node. 2 cờ hand-shake cũng được làm trễ lại 1 chu kì để đảm bảo timing chính xác cho khối memory ở phía sau. Khi cờ i_valid về 0 các ngõ ra prev_st_00 tới prev_st_11, slt_node sẽ giữ nguyên giá trị hiện tại của chúng mà không tính toán và cập nhật giá trị mới để tránh lỗi. Hoạt động và timing của khối hoàn toàn chính xác theo đặc tả.

3.1.4 Module memory

Mục tiêu: Đối chiếu hoạt động và timing của khối memory với phần đặc tả.

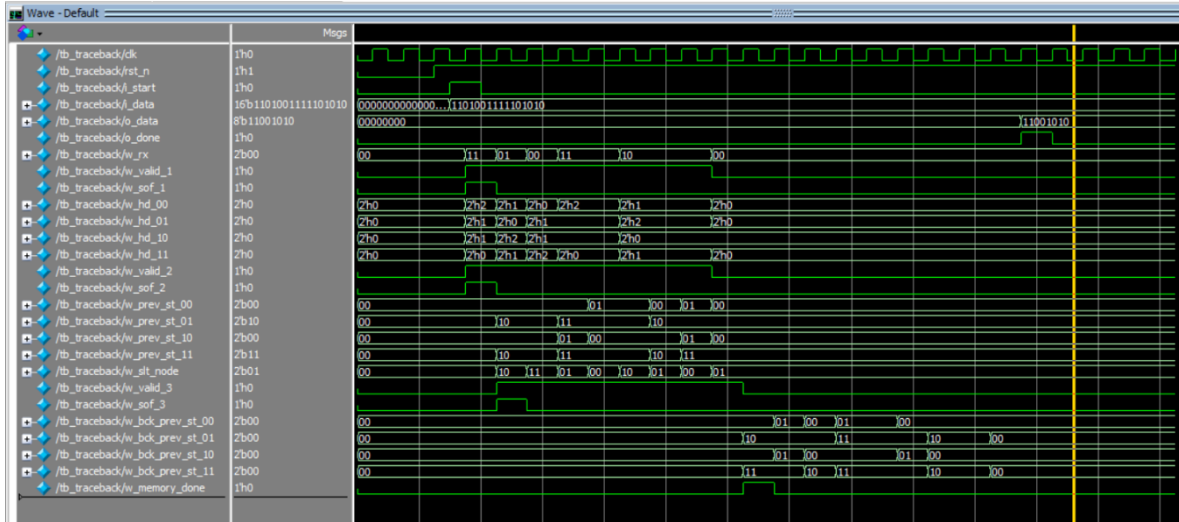


Hình 3.4: Waveform mô tả module memory

Đánh giá: Waveform chia làm 2 pha Đọc/Ghi rõ rệt. Ở pha Ghi (8 chu kì đầu), các ngõ ra bck_prev_st_00 tới bck_prev_st_11 được ép về 0. Ngay khi chuyển sang pha Đọc, mạch xuất cờ o_memory_done trong đúng 1 chu kì, các tín hiệu bck_prev_st_00 tới bck_prev_st_11 được nhả lần lượt chính xác trong 8 chu kì. Khi kết thúc pha Đọc, các ngõ ra đều được ép về 0 để tối ưu năng lượng và tránh lỗi. Hoạt động và timing của khối hoàn toàn chính xác theo đặc tả.

3.1.5 Module traceback

Mục tiêu: Đối chiếu hoạt động và timing của khối traceback với phần đặc tả.

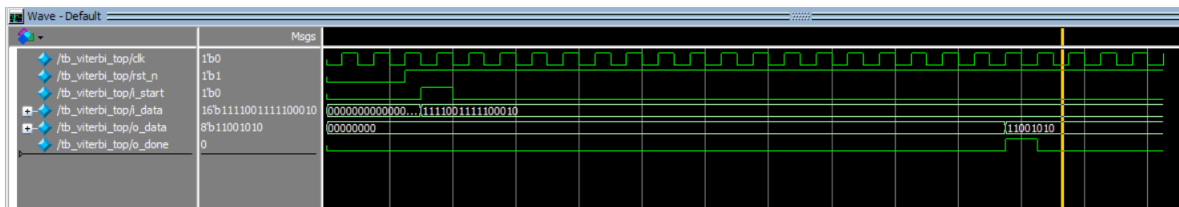


Hình 3.5: Waveform mô phỏng module traceback

Đánh giá: Waveform cho thấy module traceback đã tính toán và xuất ra được chính xác 8-bit giải mã o_data so với lý thuyết, đồng thời kéo cờ o_done lên trong 1 chu kỳ. Hoạt động và timing của khối hoàn toàn chính xác theo đặc tả.

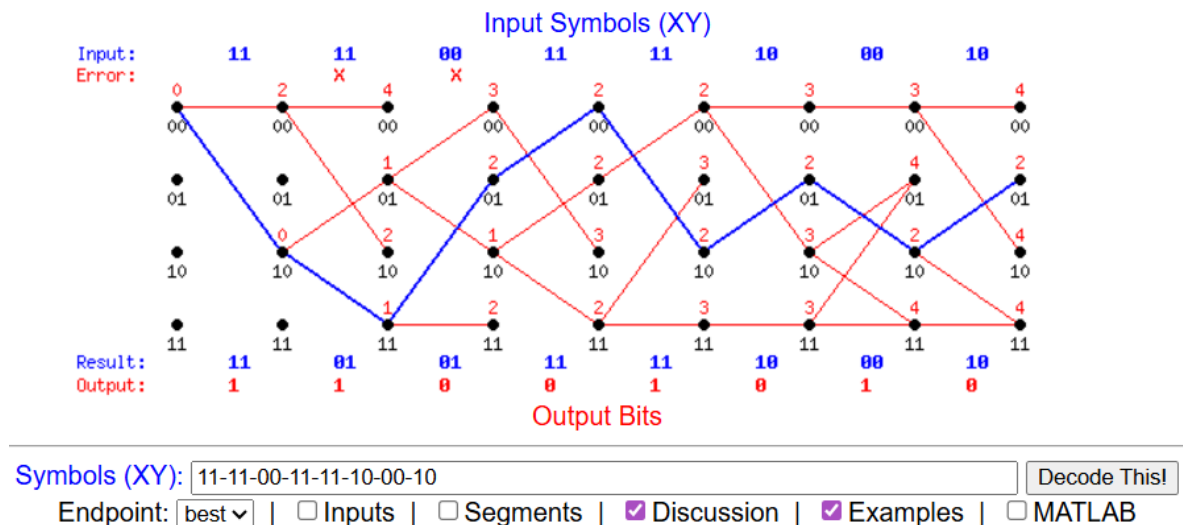
3.1.6 Module top

Mục tiêu: Kiểm tra tính chính xác của kết nối giữa các module con, kiểm tra tính chính xác, timing của bộ giải mã.



Hình 3.6: Waveform mô phỏng module top

Đánh giá: Waveform cho thấy toàn bộ hệ thống mất 19 chu kỳ clk để tính toán và xuất ra kết quả 11-00-10-10 tương ứng với input 11-11-00-11-11-10-00-10. Kết quả này so sánh với kết quả giải mã trên phần mềm EE4235 Viterbi Online là hoàn toàn tương đồng.



Hình 3.7: Kết quả trên phần mềm EE4235 Viterbi Online

3.2 Kiểm chứng toàn hệ thống với Golden Model

Mục đích: Sau khi các module đơn lẻ đã vượt qua kiểm tra sanity test, mục tiêu tiếp theo là kiểm tra tính chính xác của toàn bộ hệ thống giải mã Viterbi khi xử lý luồng dữ liệu liên tục với nhiều trường hợp đầu vào khác nhau. Việc kiểm chứng sử dụng bộ dữ liệu Golden Model được giảng viên hướng dẫn cung cấp, bao gồm 1025 trường hợp cho chuỗi 16-bit đầu vào (file input.txt) và chuỗi 8-bit đầu ra tương ứng (file output.txt), đảm bảo thiết kế có khả năng giải mã chính xác trong nhiều điều kiện dữ liệu khác nhau.

3.2.1 Kiến trúc môi trường kiểm thử

Để xử lý lượng lớn dữ liệu mà không tốn thời gian gán giá trị thủ công, môi trường kiểm thử được thiết kế với cơ chế đọc file I/O tự động:

- Testbench sử dụng các hàm `$fopen` và `$fscanf` cho phép hệ thống mở và đọc đồng thời các giá trị trong 2 file input.txt và output.txt
- Dữ liệu từ file được nạp vào các biến nội bộ ở mỗi vòng lặp `while`, cho phép hệ thống kiểm tra và in ra kết quả lần lượt từng testcase cho đến khi kết thúc file.

3.2.2 Kích bản mô phỏng và cơ chế Self – Checking

Kịch bản mô phỏng được xây dựng dựa trên nguyên tắc đồng bộ nghiêm ngặt để đảm bảo tính ổn định của hệ thống:

- **Nạp dữ liệu:** Dữ liệu đầu vào `exp_in` đọc được từ file input.txt sẽ được gán vào đầu vào `i_data` của DUT, đồng thời bật cờ `i_start` lên 1 vào

sườn xuống của clk (negedge clk) để tránh các lỗi thời gian, đảm bảo mạch luôn lấy mẫu đúng dữ liệu ở sườn lên clk tiếp theo.

- *Trích xuất và so sánh kết quả:* Testbench không sử dụng các khoảng trễ thời gian cố định mà sử dụng lệnh `wait(o_done == 1)`, điều này giúp Testbench vẫn hoạt động đúng khi kiến trúc RTL thay đổi. Ngay khi nhận được cờ hoàn tất o_done, giá trị DUT trả ra o_data sẽ được đối chiếu với kết quả exp_out đọc từ file output.txt thông qua toán tử `==`, kết quả đúng sẽ in ra PASSED, sai in ra FAILED đồng thời có 2 biến đếm pass_cnt (số case pass) fail_cnt (số case fail) sẽ tăng tương ứng với mỗi testcase PASSED/FAILED. Sau khi hoàn thành kiểm tra toàn bộ các testcase sẽ in ra tổng số case pass và fail.

3.2.3 Kết quả mô phỏng tổng thể

Quá trình mô phỏng tự động đã thực thi toàn bộ 1025 testcase 1 cách liên tục. Kết quả hiển thị trên bảng điều khiển của QuestaSim cho thấy sự khớp tuyệt đối giữa mô hình RTL và Golden Model.

Thông kê:

- Tổng số Test Case đã chạy: 1025
- Số trường hợp PASSED: 1025
- Số trường hợp FAILED: 0

Kết luận: Hệ thống đạt độ chính xác 100%, khẳng định bộ giải mã Viterbi quyết định cứng (hard-decision) đã được xây dựng hoàn toàn đúng đắn.

```
# -----  
# | CASE:          1021 |  
# PASSED | Input: 1101101010101111 | Exp: 11111100 | Act: 11111100  
# -----  
# | CASE:          1022 |  
# PASSED | Input: 1101101010100011 | Exp: 11111100 | Act: 11111100  
# -----  
# | CASE:          1023 |  
# PASSED | Input: 1101101010100101 | Exp: 11111100 | Act: 11111100  
# -----  
# | CASE:          1024 |  
# PASSED | Input: 1101101010100110 | Exp: 11111100 | Act: 11111100  
# -----  
# | CASE:          1025 |  
# PASSED | Input: 1101101010100110 | Exp: 11111100 | Act: 11111100  
# *****  
# |||| (^.^) |||| < END SIMULATION > |||| (^.^) ||||  
# *****  
# ----- TONG KET -----  
# |Tong so testcase: 1025  
# |So Case PASS      : 1025  
# |So Case FAIL      : 0  
# Break in Module testbench at D:/VLSI_viterbi/tb/testbench.v line 102
```

Hình 3.8: Kết quả mô phỏng trên QuestaSim

3.3 Đánh giá chất lượng kiểm thử (Code Coverage)

Mục đích: Nếu như việc chạy 1025 testcase chỉ chứng minh được hệ thống đúng với kịch bản đã lường trước, thì độ bao phủ (Code Coverage) là tiêu chuẩn bắt buộc để kiểm tra xem testbench đã thực sự quét qua mọi ngóc ngách của thiết kế RTL hay chưa. Mục tiêu của phần này là chứng minh phần code RTL không chứa các đoạn mã thừa và mọi nhánh logic đều đã được kiểm chứng.

3.3.1 Các tiêu chí đánh giá

Quá trình đánh giá Code Coverage trên QuestaSim tập trung vào 2 chỉ số quan trọng nhất:

- *Statment Coverage*: Đo lường tỉ lệ các dòng code RTL đã thực sự được thực thi trong quá trình mô phỏng.
- *Branch Coverage*: Đo lường tỉ lệ các nhánh rẽ điều kiện (*if else*, *case*, toán tử *?:*) đã được kiểm tra ở tất cả các nhánh chưa.

3.3.2 Đánh giá kết quả Coverage tổng thể

Dựa trên báo cáo từ QuestaSim sau khi chạy toàn bộ 1025 tập dữ liệu mẫu, thiết kế giải mã Viterbi (DUT) đã đạt được những con số lý tưởng:

- Mức độ toàn hệ thống (module viterbi_top): Đạt mức bao phủ 100% Statement (168/168 lệnh) và 100% Branch (82/82 nhánh).
- Mức độ module thành phần:
 - Module extract_bit: 100% Statement (16/16), 100% Branch (6/6).
 - Module branch_metric: 100% Statement (8/8), 100% Branch (8/8).
 - Module add_comp_slt: 100% Statment (59/59), 100% Branch (37/37).
 - Module memory: 100% Statement (58/58), 100% Branch (18/18).
 - Module traceback: 100% Statement (27/27), 100% Branch (13/13).
- Đánh giá môi trường Testbench: testbench đạt độ bao phủ 97,7% Statement và 96,6% Branch (missed 3 nhánh). Đây là kết quả hoàn toàn logic và tích cực. Các nhánh bị bỏ lỡ chính là các đoạn code báo lỗi trong cấu trúc *if else*. Việc các nhánh này không bị chạm tới minh chứng cho việc 1025/1025 testcase đều đã pass thành công.

The screenshot displays the 'Instance Coverage' window in QuestaSim. It features a table with columns for Instance, Design unit, Design unit type, Total coverage, Stmt count, Stmt hit, Stmt missed, Stmt %, Stmt graph, Branch count, Branches hit, Branches missed, Branch %, Branch graph, UDP Condition rows, UDP Conditions hit, and UDP Conditions missed. The table lists several instances of 'viterbi_top' and 'testbench' modules, all showing 100% coverage. Below the table, there is a 'sim (Recursive Coverage Aggregation)' section with a menu bar and a toolbar. The bottom part of the image shows a hierarchical tree view of the design units, including 'viterbi_top', 'testbench', and 'viterbi_capacity'.

Instance	Design unit	Design unit type	Total coverage	Stmt count	Stmt hit	Stmt missed	Stmt %	Stmt graph	Branch count	Branches hit	Branches missed	Branch %	Branch graph	UDP Condition rows	UDP Conditions hit	UDP Conditions missed
/testbench/DUT/viterbi_top	viterbi_top	Module	89.8%	16	16	0	100%		6	6	0	100%				
/testbench/DUT/viterbi_top/extract_bit	extract_bit	Module	98.5%	16	16	0	100%		6	6	0	100%				
/testbench/DUT/viterbi_top/branch_me	branch_me	Module	100%	8	8	0	100%		8	8	0	100%				
/testbench/DUT/viterbi_top/add_comp_slt	add_comp_slt	Module	93.6%	59	59	0	100%		37	37	0	100%				
/testbench/DUT/viterbi_top/memory	memory	Module	65.3%	58	58	0	100%		18	18	0	100%				
/testbench/DUT/viterbi_top/traceback	traceback	Module	97%	27	27	0	100%		13	13	0	100%				

Hình 3.9: Báo cáo Coverage trên QuestaSim

Kết luận: Với 1025 testcase đã PASSED cùng với mức Coverage đạt 100% (Statement và Branch), quá trình Functional Verification khẳng định bộ giải mã Viterbi đã được thiết kế hoàn toàn đúng đắn về mặt logic, đáp ứng các tiêu chuẩn kiểm chứng vi mạch và đã hoàn toàn sẵn sàng cho các bước tổng hợp (Synthesis) và thiết kế vật lý (Layout).

CHƯƠNG 4. THIẾT KẾ VẬT LÝ

Chương 4 tập trung vào giai đoạn hiện thực hóa thiết kế từ mức RTL sang cấu trúc vật lý thực tế trên chip (Layout). Nội dung chương sẽ trình bày chi tiết luồng thực hiện từ bước tổng hợp logic, quy hoạch mặt bằng, xây dựng cây xung nhịp (CTS), đến đi dây chi tiết và kiểm tra hoàn thiện (Sign-off). Xuyên suốt quá trình này, trọng tâm được đặt vào việc thực hiện các chiến lược tối ưu hóa để giải quyết các thách thức về trễ truyền dẫn và lỗi đua tín hiệu (Race condition), nhằm đảm bảo thiết kế hội tụ đầy đủ các chỉ số về công suất, diện tích và đặc biệt là hiệu năng tại tần số hoạt động 200MHz. Kết quả cuối cùng của chương là file GDSII hoàn chỉnh, sẵn sàng cho công đoạn sản xuất.

4.1 Mục tiêu thiết kế

Giai đoạn thiết kế vật lý nhằm hiện thực hóa bộ giải mã Viterbi từ mức công logic sang bản vẽ vật lý dựa trên thư viện công nghệ Sky130.

Các mục tiêu chính bao gồm:

- Performance (Tốc độ): Đảm bảo mạch hoạt động ổn định ở tần số 200MHz.
- Power (Công suất): Tối ưu công suất của mạch với mức tần số 200MHz
- Diện tích: Tối ưu hóa tổng diện tích các cell, duy trì mật độ sử dụng hợp lý để tránh tắc nghẽn khi đi dây
- Tính toàn vẹn: Đạt trạng thái Timing Closure, không vi phạm Setup và Hold time
- Độ tin cậy: Hoàn thiện bản vẽ GDSII không có lỗi vật lý, vượt qua các bài kiểm tra tiêu chuẩn:
 - Check DRC
 - Check Connectivity

4.2 Tổng hợp logic (Logic Synthesis)

Quá trình tổng hợp logic được thực hiện bằng công cụ Cadence Genus. Mục tiêu của bước này là chuyển đổi mã nguồn Verilog sang mức Gate-level Netlist dựa trên thư viện công nghệ Sky130, đồng thời đáp ứng các ràng buộc về thời gian

4.2.1 Thiết lập môi trường và ràng buộc

Để đạt được chất lượng thiết kế đáp ứng các mục tiêu cho bộ giải mã Viterbi, quá trình tổng hợp được thiết lập với các cấu hình thông qua file `genus.tcl` và `constraints.sdc`:

Thiết lập scripts thông qua file genus.tcl:

```
set_db init_lib_search_path ../lib/
set_db init_hdl_search_path ../rtl/

read_libs sky130_ss_1.62_125_nldm.lib
read_hdl {extract_bit.v branch_metric.v add_comp_slt.v memory.v traceback.v viterbi_top.v}

elaborate viterbi_top

read_sdc ../constraints/constraints.sdc

set_db [get_db lib_cells *SDFF*] .dont_use true

set_db syn_generic_effort high
set_db syn_map_effort high
set_db syn_opt_effort high

syn_generic
syn_map
syn_opt

# Report
report_timing > reports/report_timing.rpt
report_area > reports/report_area.rpt
report_power > reports/report_power.rpt
report_gates > reports/report_gates.rpt
report_qor > reports/report_qor.rpt

# Output
write_hdl > outputs/viterbi_netlist.v
write_sdc > outputs/viterbi_sdc.sdc
```

Hình 4.1: Nội dung file genus.tcl

Phân tích:

- Thư viện công nghệ: Nhóm sử dụng thư viện sky130_ss_1.62_125_nldm.lib (Hoạt động Slow-Slow, nhiệt độ 125°C). Đây là kiểm tra trường hợp xấu nhất (Worst-case corner) nhằm đảm bảo mạch hoạt động an toàn trong mọi điều kiện môi trường
- Chiến lược tối ưu hóa: Các mức tối ưu hóa syn_generic, syn_map và syn_opt đều được thiết lập ở mức high để ép công cụ Genus tìm ra kiến trúc logic nhanh nhất và nhỏ nhất có thể.
- Loại trừ Cell không cần thiết: Lệnh set_db [get_db lib_cells *SDFF*] .dont_use được sử dụng để ngăn công cụ tổng hợp chọn các Flip-Flop có hỗ trợ quét (Scan DFF), giúp tiết kiệm diện tích và công suất
- Các file báo cáo về các thông số được lưu vào folder reports, 2 file viterbi_netlist.v và viterbi_sdc.sdc được lưu vào folder outputs phục vụ giai đoạn Layout sau đó

Thiết lập ràng buộc SDC thông qua file constraints.sdc:

```
# 1. Clock
create_clock -name clk -period 4.5 [get_ports clk]

# 2. Clock Uncertainty
set_clock_uncertainty 0.3 [get_clocks clk]

# 3. Input Delay
set_input_delay -clock clk 1.0 [remove_from_collection [all_inputs] [get_ports clk]]

# 4. Input Transition
set_input_transition 0.1 [remove_from_collection [all_inputs] [get_ports clk]]

# 5. Output Delay
set_output_delay -clock clk 1.0 [all_outputs]

# 6. Load
set_load 0.1 [all_outputs]
~
~
~
```

Hình 4.2: Nội dung file constraints.sdc

Phân tích:

- Chiến lược Over-constraining: Dù mục tiêu đề ra là 200MHz, xung nhịp clk được ép xuống chu kỳ 4.5ns (222 MHz). Khoảng dư 0.5ns này được dùng làm biên an toàn bù đắp cho trễ ở bước Layout
- Độ bất định xung nhịp: Thiết lập ở mức 0.3ns để mô phỏng các sai số về Jitter và Skew thực tế.
- Ràng buộc I/O: Khai báo trễ đầu vào (set_input_delay) và trễ đầu ra (set_output_delay) đều là 1.0ns, giúp định hướng cho công cụ tối ưu các đường dữ liệu giao tiếp với môi trường ngoài

4.2.2 Kết quả tổng hợp

Sau khi chạy tiến trình tổng hợp với các ràng buộc trên, báo cáo cho thấy thiết kế đã hội tụ thành công về các thông số PPA.

A. Đánh giá Timing

- Báo cáo Timing cho thấy thiết kế đạt trạng thái an toàn (MET) với Slack dương +40 ps tại chu kỳ 4.5ns
- Đường trễ tới hạn (Critical Path) xuất phát từ thanh ghi của khối extract_bit (u_extract_o_sof_reg/CK) và kết thúc tại thanh ghi của khối add_comp_slt (u_acs_pm_01_reg[4]/D)
- Tổng trễ đường truyền dữ liệu (Data Path) là 3885 ps. Việc hệ thống đáp ứng được chu kỳ 4.5ns ở mức logic chứng tỏ thiết kế hoàn toàn sẵn sàng để đạt mức 200MHz ở giai đoạn Layout

```

=====
Path 1: MET (40 ps) Setup Check with Pin u_acs_pm_01_reg[4]/CK->D
Group: clk
Startpoint: (R) u_extract_o_sof_reg/CK
Clock: (R) clk
Endpoint: (R) u_acs_pm_01_reg[4]/D
Clock: (R) clk

          Capture          Launch
Clock Edge:+      4500          0
Src Latency:+      0          0
Net Latency:+      0 (I)      0 (I)
Arrival:=      4500          0

Setup:-      275
Uncertainty:-  300
Required Time:= 3925
Launch Clock:-  0
Data Path:-    3885
Slack:=      40

```

Hình 4.3: Report Timing sau bước Synthesis

B. Đánh giá Area

- Tổng số lượng cell tiêu thụ (Cell-Count) là 644 cổng logic.
- Tổng diện tích mức logic (Total Cell Area) đạt $9815.201\mu m^2$. Kết quả này phản ánh đúng độ phức tạp của một bộ giải mã Viterbi có chứa các khối toán học (Add-Compare-Select) và hệ thống cấp phát bộ nhớ.

```

=====
Generated by:      Genus(TM) Synthesis Solution 23.17-s085_1
Generated on:      May 14 2026 03:51:10 pm
Module:           viterbi_top
Operating conditions: ss_1.62_125 (balanced_tree)
Wireload mode:    enclosed
Area mode:        timing library
=====

Instance  Module  Cell-Count  Cell-Area  Net-Area  Total-Area  Wireload
-----
viterbi_top NA          644    9815.201    0.000    9815.201 <none> (D)
(D) = wireload is default in technology library

```

Hình 4.4: Report Area sau bước Synthesis

C. Đánh giá Power

Tổng công suất tiêu thụ toàn mạch ở chế độ hoạt động bình thường là khoảng 2.35mW, được phân bố như sau:

- Công suất tĩnh (Leakage Power): Vô cùng nhỏ, chiếm 0.05%, cho thấy thư viện công nghệ Sky130 kiểm soát dòng rò rất tốt
- Công suất động (Dynamic Power): Chiếm 99.95% tổng công suất, bao gồm công suất chuyển mạch (Switching – 32.53%) và công suất nội bộ (Internal – 67.42%)

- Đi sâu vào phân tích các thành phần, Thanh ghi (Register) là phần tử tiêu tốn nhiều năng lượng nhất (chiếm 60.39%), tiếp theo là khối tổ hợp Logic (chiếm 34.17%). Điều này hoàn toàn phù hợp với đặc thù của bộ giải mã Viterbi vốn cần rất nhiều Flip-Flop để lưu trữ các trạng thái đường đi

```
=====
Generated by:      Genus(TM) Synthesis Solution 23.17-s085_1
Generated on:      May 14 2026 03:51:10 pm
Module:            viterbi_top
Operating conditions: ss_1.62_125 (balanced_tree)
Wireload mode:     enclosed
Area mode:         timing library
=====

  Instance  Module  Cell-Count  Cell-Area  Net-Area  Total-Area  Wireload
-----
viterbi_top NA          644    9815.201      0.000    9815.201 <none> (D)
(D) = wireload is default in technology library
```

Hình 4.5: Report Power sau bước Synthesis

4.3 Thiết kế vật lý (Layout)

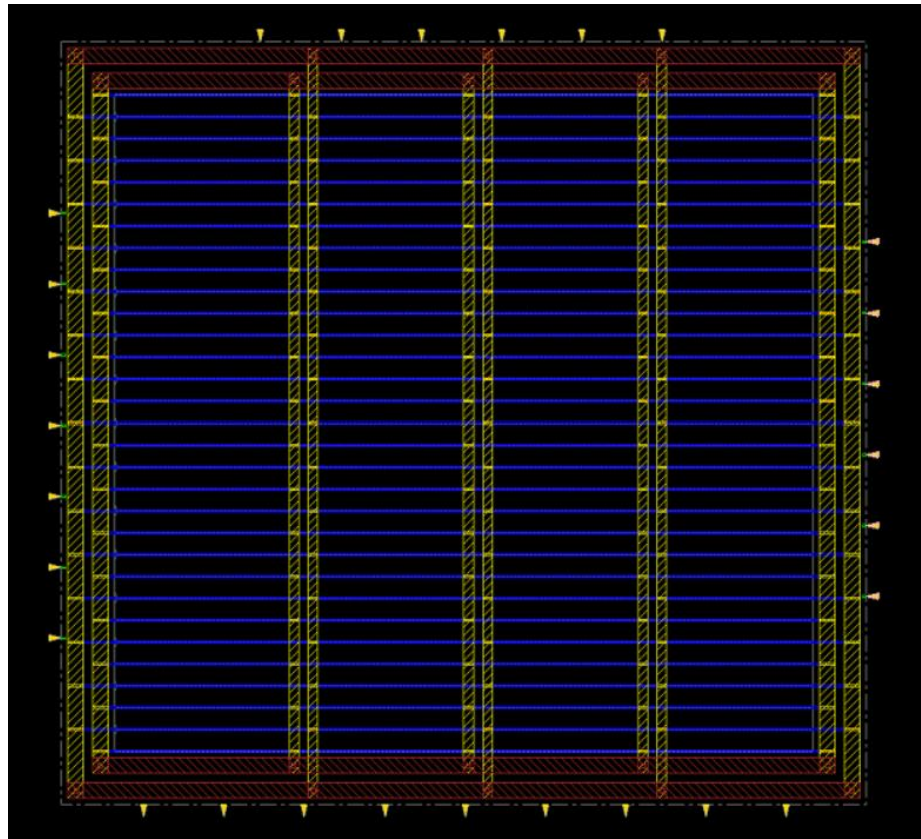
Quá trình thiết kế vật lý được thực hiện trên nền tảng Cadence Innovus, nhằm chuyển đổi Netlist mức cổng thành một bản vẽ hình học hoàn chỉnh có khả năng chế tạo. Mạch được triển khai theo quy trình chuẩn từ dưới lên (Bottom-up), bao gồm các công đoạn sau:

4.3.1 Khởi tạo và Quy hoạch Floorplanning & Power Planning

Khởi tạo thiết kế: Nạp dữ liệu file viterbi_netlist.v, file ràng buộc và các thư viện vật lý (lef) của công nghệ Sky130 vào hệ thống

Quy hoạch Floorplan: Thiết lập ranh giới diện tích chip (Die Area) và vùng lõi chứa cell (Core Area). Các cổng giao tiếp (I/O Pins) được phân bố hợp lý xung quanh viền để tối ưu quãng đường tín hiệu đi vào mạch

Quy hoạch Power Planning: Nhóm thiết kế mạng lưới cấp điện VDD và VSS dạng vòng (Power Rings) bao quanh lõi, kết hợp với các lưới Power Stripes đan xen giữa các lớp kim loại cấp cao để tận dụng đặc tính điện trở thấp của chúng



Hình 4.6: Mạch sau bước quy hoạch Floorplan và Powerplan

4.3.2 Sắp xếp và tối ưu hóa sơ bộ (Placement & Pre-CTS)

Giai đoạn này sẽ được công cụ Cadence Innovus thực hiện tự động thông qua các dòng lệnh hoặc nút bấm GUI. Innovus tiến hành đặt tự động hàng trăm standard cells và các hàng đã được định nghĩa sẵn trong Core

Quá trình sắp xếp được điều hướng bởi các thuật toán tối ưu hóa nhằm hai mục đích chính: (1) Giảm thiểu tổng chiều dài dây dẫn (Wirelength) để tiết kiệm điện năng và (2) Tránh đặt các cell quá sát nhau tại một khu vực gây tắc nghẽn cục bộ (Congestion). Ở giai đoạn này thiết kế vẫn đang giữ ở mức ràng buộc $T = 4.5ns$ để hệ thống ưu tiên đẩy nhanh các đường trễ nhất

4.3.3 Tổng hợp cây xung nhịp (Clock Tree Synthesis – CTS)

Tín hiệu Clock là tín hiệu di chuyển qua nhiều linh kiện nhất và đi xa nhất. Do đó mục tiêu của bước CTS là phân phối xung nhịp từ cổng gốc (Clock source) đến chân Clock của toàn bộ các thanh ghi sao cho tín hiệu đến đích cùng một lúc.

Công cụ tự động xây dựng cấu trúc dạng cây, chèn thêm các Clock Buffers và Inverters tại các điểm rẽ nhánh để cân bằng tải và giảm thiểu độ lệch pha

4.3.4 Đi dây chi tiết và Tối ưu (Routing & Post – Route)

Đi dây tín hiệu (Routing): Thuật toán NanoRoute được sử dụng để kết nối vật lý hàng nghìn chân tín hiệu với nhau, tuân thủ nghiêm ngặt các quy tắc thiết kế (Design Rules) của Sky130 trên các lớp kim loại (từ met1 đến met5)

Cập nhật xung nhịp và Tối ưu hóa Hold Time:

- Sau khi hoàn tất đi dây và nắm được chính xác thông số trễ vật lý (RC Delay), nhóm đã tiến hành gỡ bỏ ràng buộc, trả chu kỳ đồng hồ về đúng mục tiêu đề ra là $T = 5.0ns$ (200 MHz)
- Việc đi dây thực tế đã làm phát sinh một số lỗi đua tín hiệu (Race condition), thể hiện qua các vi phạm Hold Time. Lệnh `opt_design -post_route -hold` được kích hoạt để khắc phục. Công cụ đã dò tìm các đường truyền dữ liệu đi quá nhanh (Fast paths) và tự động chèn thêm các cổng đệm tạo trễ (Delay Buffers), giúp tín hiệu bị giữ lại đủ lâu để thanh ghi đích kịp ghi nhận trạng thái

4.3.5 Hoàn thiện và Kiểm tra vật lý (Sign-off & Physical Verification)

Sau khi khắc phục xong các vi phạm Hold Time, nhóm hoàn thiện và chạy các lệnh kiểm tra rồi xuất file bản vẽ GDSII

- Chèn Filler Cells: Để đảm bảo tính liên tục của các giếng N-well/P-well và đáp ứng yêu cầu về mật độ kim loại của nhà máy sản xuất, các Filler cells (cell lấp chỗ trống không có chức năng logic) đã được chèn vào mọi khoảng trống còn lại trong vùng Core
- Kiểm tra Physical Verification: Chạy các lệnh kiểm tra cuối cùng bao gồm `check_drc` và `check_connectivity` để đảm bảo mạch không có hiện tượng chập điện, đứt gãy kết nối hay vi phạm khoảng cách
- Stream Out: Xuất bản vẽ cuối cùng dưới định dạng cơ sở dữ liệu hình học GDSII, chính thức khép lại quy trình thiết kế vật lý

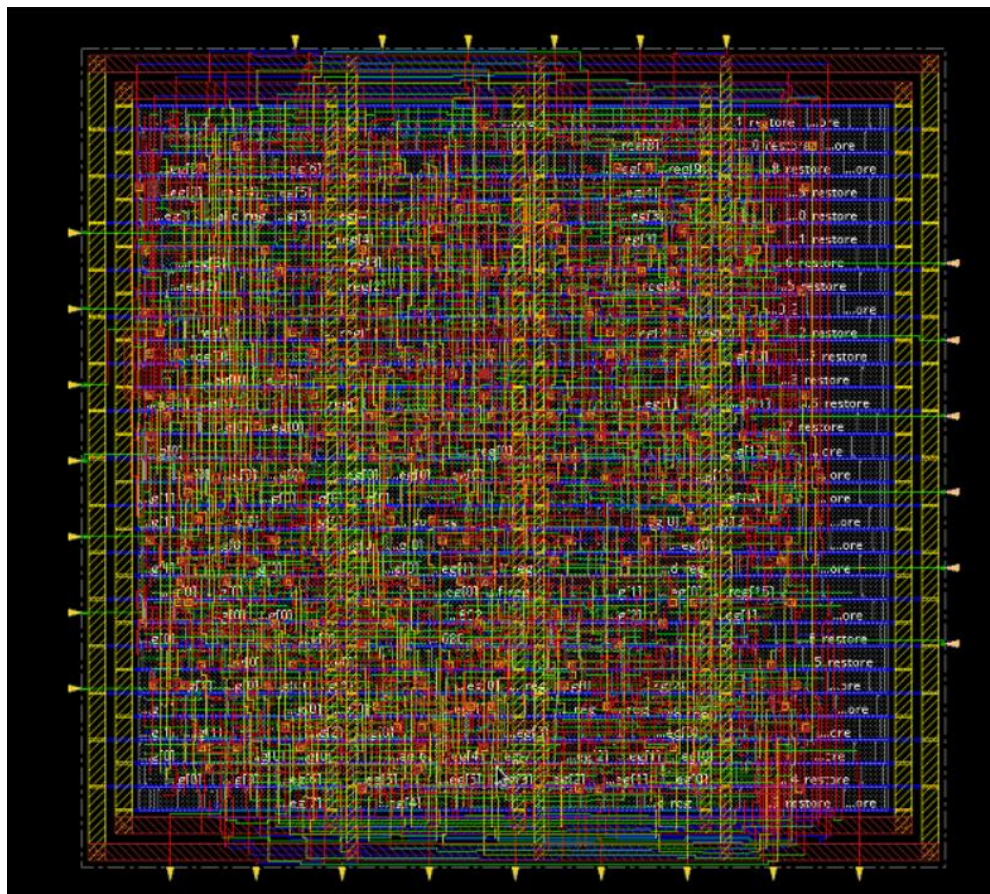
```
#####
# Generated by:      Cadence Innovus 23.37-s090.1
# OS:               Linux x86_64(Host ID iclab)
# Generated on:      Thu May 14 18:06:45 2026
# Design:           viterbi_top
# Command:          check_drc -out_file reports/report_drc.rpt
#####
No DRC violations were found
```

Hình 4.7: Kết quả `check_drc`

```
#####
# Generated by:      Cadence Innovus 23.37-s090_1
# OS:               Linux x86_64(Host ID iclab)
# Generated on:      Thu May 14 18:07:01 2026
# Design:           viterbi_top
# Command:          check_connectivity -out_file reports/report_connectivity.rpt
#####
Verify Connectivity Report is created on Thu May 14 18:07:01 2026
```

```
Begin Summary
  Found no problems or warnings.
End Summary
```

Hình 4.8: Kết quả check_connectivity



Hình 4.9: Mạch sau khi hoàn tất Layout

4.4 Phân tích kết quả và đánh giá

Sau khi hoàn tất toàn bộ quy trình Layout, các báo cáo Sign – off cho thất các thông số PPA của thiết kế

4.4.1 Đánh giá Timing

Báo cáo report_timing xác nhận thiết kế đã đáp ứng mức tần số mục tiêu 200MHz với độ tin cậy cao:

Trạng thái: MET (không vi phạm Setup Time)

Setup Slack: Đạt giá trị dương +0.268 ns tại góc mô phỏng xấu nhất (Worst-Case). Điều này chứng tỏ mạch có một biên độ an toàn cực tốt, sẵn sàng chống chịu được các biến thiên về nhiệt độ và điện áp khi sản xuất thực tế

Phân tích đường trễ tới hạn (Critical Path): Đường dẫn giới hạn tốc độ của toàn mạch bắt đầu từ thanh ghi extract_bit (u_extract_o_valid_reg/CK) và kết thúc tại thanh ghi của khối ACS (u_acs_pm_10_reg[4]/D)

```
#####
# Generated by: Cadence Innovus 23.37-s090_1
# OS: Linux x86_64(Host ID iclab)
# Generated on: Thu May 14 18:06:06 2026
# Design: viterbi_top
# Command: report_timing > reports/report_timing.rpt
#####
Path 1: MET (0.268 ns) Setup Check with Pin u_acs_pm_10_reg[4]/CK->D
View: wc
Group: clk
Startpoint: (R) u_extract_o_valid_reg/CK
Clock: (R) clk
Endpoint: (R) u_acs_pm_10_reg[4]/D
Clock: (R) clk

      Capture      Launch
Clock Edge:+ 5.000 0.000
Src Latency:+ 0.000 0.000
Net Latency:+ 0.211 (P) 0.204 (P)
Arrival:= 5.211 0.204

      Setup:- 0.122
Uncertainty:- 0.300
Required Time:= 4.788
Launch Clock:= 0.204
Data Path:+ 4.317
Slack:= 0.268
```

Hình 4.10: Report Timing sau Layout

4.4.2 Đánh giá Area

So sánh báo cáo report_area ở giai đoạn này với bước Synthesis, thiết kế đã có sự thay đổi về quy mô vật lý:

- Số lượng linh kiện: Tăng từ 644 lên 868 instances. Sự gia tăng này là hệ quả tất yếu và hợp lý của quá trình Physical Design, do công cụ Innovus đã chèn thêm các Clock Buffers (trong bước CTS), các Delay Buffers

(để sửa lỗi Hold), và các Physical Cells (như Filler, Tap cells) để phục vụ sản xuất

- Tổng diện tích (Total Area): Đạt $13,077.485\mu m^2$. Mặc dù lớn hơn so với diện tích mức cổng thuần túy ($9815\mu m^2$). đây là diện tích vật lý thực tế đã bao gồm không gian trống để đi dây và lưới điện. Mức diện tích này phản ánh đúng mật độ tiêu chuẩn của công nghệ Sky130 đối với một mạch xử lý tín hiệu số phức tạp như Viterbi

Hinst Name	Module Name	Inst Count	Total Area
viterbi_top		868	13077.485

Hình 4.11: Report Area sau Layout

4.4.3 Đánh giá Power

Báo cáo report_power tại điện áp VDD = 1.62V và tần số 200 MHz cho thấy tổng công suất tiêu thụ của chip cực kỳ tối ưu, đạt mức 2.327 mW:

- Phân bố theo tính chất:
 - Internal Power (Công suất nội bộ): Chiếm tỷ trọng cao nhất (55.56%, tương đương 1.292 mW), sinh ra do dòng điện ngắn mạch bên trong các cell khi chuyển trạng thái
 - Switching Power (Công suất chuyển mạch): Chiếm 44.37% (1.032 mW), sinh ra do quá trình nạp/xả các tụ ký sinh trên mạng lưới dây dẫn mà quá trình Routing tạo ra
 - Leakage Power (Công suất rò): Vẫn duy trì ở mức cực kỳ thấp, chỉ 0.07% (0.0016 mW), cho thấy transistor của Sky130 được cách ly rất tốt
- Phân bố theo nhóm linh kiện
 - Các mạch tuần tự (Sequential) tiêu tốn nhiều năng lượng nhất (43.25%), mạch tổ hợp (Combinational) 42.61%
 - Đáng chú ý, mạng phân phối xung nhịp (Clock Tree) tiêu thụ 14.14% (0.329 mW). Tỷ lệ này chứng tỏ bước CTS đã được thực hiện rất hiệu quả, không bị "phình to" số lượng buffer gây lãng phí điện năng

Total Power

Total Internal Power:	1.29266759	55.5596%
Total Switching Power:	1.03230618	44.3691%
Total Leakage Power:	0.00165904	0.0713%
Total Power:	2.32663280	

Group	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)
Sequential	0.831	0.1743	0.0009205	1.006	43.25
Macro	0	0	0	0	0
IO	0	0	0	0	0
Combinational	0.3629	0.6277	0.0007264	0.9914	42.61
Clock (Combinational)	0.09868	0.2303	1.215e-05	0.329	14.14
Clock (Sequential)	0	0	0	0	0
Total	1.293	1.032	0.001659	2.327	100

Rail	Voltage	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)
VDD	1.62	1.293	1.032	0.001659	2.327	100

Clock	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)
clk	0.09868	0.2303	1.215e-05	0.329	14.14
Total	0.09868	0.2303	1.215e-05	0.329	14.14

Hình 4.12: Report Power sau Layout

4.5 Tổng kết

Chương 4 đã trình bày chi tiết và hoàn chỉnh quy trình thiết kế vật lý (Physical Design) cho bộ giải mã Viterbi sử dụng thư viện công nghệ Sky130. Bằng việc áp dụng luồng thiết kế chuẩn công nghiệp trên công cụ Cadence Innovus, cấu trúc mức cổng (Netlist) đã được chuyển đổi thành công sang bản vẽ GDSII

Bảng 4.1: Tổng hợp thông số PPA và kết quả Sign – off sau quá trình Layout

Thông số / Chỉ tiêu	Kết quả đạt được	Đánh giá
Tần số hoạt động	200 MHz	Đạt
Setup Slack (Worst – Case)	Setup Slack	Đạt
Hold Slack	Dương (> 0 ns)	Đạt
Tổng diện tích	13,077.485 μm^2	Chưa đạt
Số lượng linh kiện	868	Đạt
Tổng công suất	2.327 mW	Chưa đạt
Kiểm tra DRC	0 Violations	Đạt
Kiểm tra Connectivity	0 Violations	Đạt
Sản phẩm đầu ra	File viterbi_final.gds	Đã xuất thành công

Đánh giá chung:

Bộ giải mã Viterbi đã được thiết kế Layout thành công. Thiết kế đã vượt qua các bài kiểm tra Timing khắc nghiệt nhất ở tần số 200MHz, file bản vẽ GDSII cuối cùng đã sẵn sàng cho quá trình Tape – out.

CHƯƠNG 5. KẾT LUẬN

5.1 Kết quả đạt được

Sau một thời gian tập trung nghiên cứu, thiết kế, kiểm chứng và triển khai vật lý, đề tài “Thiết kế bộ giải mã sử dụng thuật toán Viterbi 16-bit” đã hoàn thành đầy đủ các mục tiêu đề ra theo quy trình luồng thiết kế vi mạch chuẩn công nghiệp (ASIC Design Flow). Các kết quả cụ thể bao gồm:

Về mặt Lý thuyết và Kiến trúc:

- Nghiên cứu và làm chủ thuật toán giải mã Viterbi với các tham số cấu hình: coding rate $R = 1/2$, constraint $K = 3$, đa thức sinh $x^2 + x + 1$ và $x^2 + 1$
- Đề xuất và hiện thực hóa thành công kiến trúc Pipelined Dataflow mức RTL bằng ngôn ngữ Verilog, chia cấu trúc hệ thống thành các module chức năng độc lập, tường minh: `extract_bit`, `branch_metric`, `add_comp_sl`, `memory`, và `traceback`

Về mặt Kiểm chứng Chức năng:

- Xây dựng môi trường Testbench tự động hóa mạnh mẽ trên công cụ QuestaSim, thực hiện kiểm tra chính xác toàn bộ kịch bản lỗi với 1025 tập dữ liệu mẫu từ file cấu hình đầu vào.
- Đạt chỉ số độ bao phủ mã (Code Coverage) lý tưởng: 100% Statement (168/168 lệnh) và 100% Branch (82/82 nhánh) đối với module toàn cục (`viterbi_top`). Kết quả này minh chứng cho tính đúng đắn tuyệt đối của logic giải mã trước khi tiến hành tổng hợp.

Về mặt Thiết kế Vật lý:

- Triển khai thành công luồng Layout từ Netlist mức cổng đến bản vẽ GDSII cuối cùng bằng công cụ Cadence Innovus trên nền tảng thư viện công nghệ Sky130
- Bản vẽ layout hoàn thiện đạt tiêu chí Sign-off, sạch hoàn toàn các lỗi về kiểm tra luật thiết kế (DRC) và lỗi kết nối hình học (Connectivity check)

Về mặt Thông số Kỹ thuật (PPA):

- Hiệu năng (Performance): Hệ thống đạt tần số hoạt động 200 MHz. Các thông số về thời gian (Timing) được đảm bảo an toàn tuyệt đối với Setup Slack đạt yêu cầu và Hold Slack duy trì giá trị dương (> 0 ns).

5.2 Những hạn chế và bài học kinh nghiệm

Bên cạnh những kết quả đã đạt được, thiết kế hiện tại vẫn tồn tại những điểm hạn chế:

Vấn đề tối ưu Diện tích (Area):

- Tổng diện tích sau layout thực tế đạt $13,077.485 \mu\text{m}^2$ (với tổng số 868 standard cells). Thông số này chưa thỏa mãn mục tiêu tối đa ban đầu

Vấn đề Tiêu thụ Công suất (Power):

- Do kiến trúc xử lý song song và hoạt động ở tần số cao, công suất tiêu thụ tổng thể của mạch (gồm công suất động và công suất rò rỉ) vượt mức chỉ tiêu đề ra ban đầu. Thiết kế chưa tích hợp sâu các kỹ thuật quản lý năng lượng nâng cao ở mức vật lý.

Bài học kinh nghiệm:

- Nhóm đã tích lũy được tư duy phân tích hệ thống sâu sắc, hiểu rõ mối quan hệ ràng buộc và sự đánh đổi giữa 3 yếu tố Công suất - Hiệu năng - Diện tích (PPA).
- Thành thạo kỹ năng thiết kế RTL và kiểm thử phần cứng bằng ngôn ngữ Verilog, sử dụng các công cụ EDA chuyên nghiệp (QuestaSim, Cadence Genus, Innovus), kỹ năng viết mã script tự động hóa luồng chạy vật lý và kỹ năng đọc/phân tích các file báo cáo tổng hợp (Report Timing, Report Power, Report Area).

5.3 Hướng phát triển của đề tài

Để cải tiến bộ giải mã Viterbi đạt mức độ tối ưu cao hơn và tiệm cận gần hơn với các ứng dụng thương mại thực tế, các hướng nghiên cứu và phát triển tiếp theo của đề tài bao gồm:

Tối ưu hóa kiến trúc RTL giảm diện tích:

- Nghiên cứu chuyển đổi hoặc lai hóa kiến trúc sang dạng tuần tự hóa trạng thái (State-serialized Viterbi) hoặc áp dụng kỹ thuật chia sẻ tài nguyên phần cứng (Resource Sharing) cho khối tính toán độ đo cộng - so sánh - chọn (ACS). Giải pháp này chấp nhận đánh đổi một phần băng thông để thu hẹp diện tích lõi (Core Area)

Áp dụng các kỹ thuật thiết kế công suất thấp:

- Tích hợp cơ chế cô lập xung nhịp tự động (Clock Gating) trực tiếp từ bước tổng hợp logic mức RTL để ngắt xung nhịp các khối thanh ghi bộ nhớ khi không có dữ liệu thay đổi, giảm thiểu tối đa công suất động (Dynamic Power).

- Sử dụng thư viện đa điện áp ngưỡng (Multi-VT Cell) nếu công nghệ cho phép, nhằm ứng dụng các cell có điện áp ngưỡng cao (High-VT) trên các đường truyền không thuộc đường găng (Non-critical paths) để triệt tiêu dòng rò rỉ (Leakage Power).

Mở rộng tính năng hệ thống:

- Nâng cấp bộ giải mã từ giải mã cứng (Hard-decision) hiện tại sang kiến trúc giải mã mềm (Soft-decision), tăng số bit lượng hóa đầu vào để cải thiện đáng kể phẩm chất hệ thống (Coding Gain) và nâng cao khả năng chống nhiễu trên các kênh truyền có tỷ lệ tín hiệu trên nhiễu (SNR) thấp.
- Mở rộng tham số cấu hình của bộ giải mã để hỗ trợ các chiều dài ràng buộc lớn hơn ($K=7$ hoặc $K=9$) theo các tiêu chuẩn truyền thông hiện đại như GSM, CDMA hoặc vệ tinh.

TÀI LIỆU THAM KHẢO

- [1] Cadence Design Systems, “Innovus Text Command Reference,” Product Version 23.17, Jan. 2026.
- [2] Cadence Design Systems, “Innovus Stylus Common UI User Guide,” Product Version 23.17, Jan. 2026.
- [3] UConn HKN, “Digital Communications: Viterbi Algorithm,” YouTube video, 26:27, 2017.
- [4] MIT, “Viterbi Decoding of Convolutional Codes,” MIT 6.02 Draft Lecture Notes, Lecture 9, Fall 2010, Oct. 2010.
- [5] University of New Brunswick, “Online Viterbi Decoding Tool ($R=1/2$, $K=3$).” [Online]. Available: ee.unb.ca/cgi-bin/tervo/viterbi2.pl

PHỤ LỤC

Code RTL:

1. Module extract_bit:

```
module extract_bit (  
    input wire clk,  
    input wire rst_n,  
    input wire i_start,  
    input wire [15:0] i_data,  
    output wire [1:0] o_rx,  
    output reg o_valid, // Báo hiệu o_rx đang chứa dữ liệu chuẩn  
    output reg o_sof // Báo hiệu cặp bit đầu tiên của chuỗi  
);  
    reg [15:0] shift_reg;  
    reg [2:0] count; // Tự đếm số lần dịch  
  
    assign o_rx = o_valid ? shift_reg[15:14] : 2'b00; // Dùng 1 thanh ghi dịch để  
    xuất các cặp bit  
  
    always @(posedge clk or negedge rst_n) begin  
        if (!rst_n) begin  
            shift_reg <= 16'd0;  
            count    <= 3'd0;  
            o_valid   <= 1'b0;  
            o_sof    <= 1'b0;  
        end else if (i_start) begin  
            // Nhận chuỗi mới  
            shift_reg <= i_data;  
            count    <= 3'd7; // Còn 7 lần dịch nữa  
            o_valid   <= 1'b1;  
            o_sof    <= 1'b1; // Cờ báo cặp bit đầu lên 1 nhịp  
        end else if (count > 0) begin  
            // Đang trong quá trình dịch  
            shift_reg <= {shift_reg[13:0], 2'b00};  
        end  
    end
```

```

        count    <= count - 1;
        o_valid  <= 1'b1;
        o_sof    <= 1'b0; // Tắt vì không phải cặp bit đầu
    end else begin
        // Trạng thái nghỉ, tắt 2 cờ
        o_valid  <= 1'b0;
        o_sof    <= 1'b0;
    end
end
endmodule

```

2. Module branch_metric:

```

module branch_metric (
    input  wire rst_n,

    // Tín hiệu Hand-shake nhận từ khối extract_bit phía trước
    input  wire i_valid,
    input  wire i_sof,
    input  wire [1:0] i_rx,

    // Khoảng cách Hamming
    output wire [1:0] hd_00,
    output wire [1:0] hd_01,
    output wire [1:0] hd_10,
    output wire [1:0] hd_11,

    // Chuyển tiếp tín hiệu Hand-shake cho khối ACS phía sau
    output wire o_valid,
    output wire o_sof
);

// Tính khoảng cách Hamming dùng XOR từng bit và cộng tổng lại
wire [1:0] dist_00 = (i_rx[1] ^ 1'b0) + (i_rx[0] ^ 1'b0);
wire [1:0] dist_01 = (i_rx[1] ^ 1'b0) + (i_rx[0] ^ 1'b1);
wire [1:0] dist_10 = (i_rx[1] ^ 1'b1) + (i_rx[0] ^ 1'b0);

```

```

wire [1:0] dist_11 = (i_rx[1] ^ 1'b1) + (i_rx[0] ^ 1'b1);

// Cho phép xuất output khi hệ thống không bị reset và dữ liệu hợp lệ
wire active = rst_n & i_valid;

assign hd_00 = active ? dist_00 : 2'b00;
assign hd_01 = active ? dist_01 : 2'b00;
assign hd_10 = active ? dist_10 : 2'b00;
assign hd_11 = active ? dist_11 : 2'b00;

// Tín hiệu Hand-shake gửi sang khối ACS
assign o_valid = active;
assign o_sof = active ? i_sof : 1'b0;
endmodule

```

3. Module add_comp_slt:

```

module add_comp_slt (
    input  wire clk,
    input  wire rst_n,
    input  wire i_valid,
    input  wire i_sof,
    input  wire [1:0] hd_00, hd_01, hd_10, hd_11,

    // Dữ liệu xuất ra cho memory và traceback
    output reg [1:0] prev_st_00, prev_st_01, prev_st_10, prev_st_11,
    output reg [1:0] slt_node,

    // Tín hiệu Hand-shake xuất ra cho memory
    output reg o_valid,
    output reg o_sof
);

// Thanh ghi lưu PM nội bộ
reg [4:0] pm_00, pm_01, pm_10, pm_11;

```

```

// Nếu i_sof = 1 -> Chu kì đầu, giá trị khởi tạo của PM 4 trạng thái: 0, 23, 23, 23 (Spec)
// Nếu i_sof = 0 -> Dừng điểm PM cũ đang lưu trong thanh ghi
wire [4:0] base_00 = i_sof ? 5'd0 : pm_00;
wire [4:0] base_01 = i_sof ? 5'd23 : pm_01;
wire [4:0] base_10 = i_sof ? 5'd23 : pm_10;
wire [4:0] base_11 = i_sof ? 5'd23 : pm_11;

// BƯỚC 1: ADD (Theo 4 PT 2-1 tới 2-4 trong Spec)
wire [4:0] c_00_from_00 = base_00 + hd_00;
wire [4:0] c_00_from_01 = base_01 + hd_11;

wire [4:0] c_10_from_00 = base_00 + hd_11;
wire [4:0] c_10_from_01 = base_01 + hd_00;

wire [4:0] c_01_from_10 = base_10 + hd_10;
wire [4:0] c_01_from_11 = base_11 + hd_01;

wire [4:0] c_11_from_10 = base_10 + hd_01;
wire [4:0] c_11_from_11 = base_11 + hd_10;

// BƯỚC 2: COMPARE & SELECT (Chọn đường đi cho từng State)
wire [4:0] sum00 = (c_00_from_00 <= c_00_from_01) ? c_00_from_00 : c_00_from_01;
wire [1:0] src_00 = (c_00_from_00 <= c_00_from_01) ? 2'b00 : 2'b01;

wire [4:0] sum10 = (c_10_from_00 <= c_10_from_01) ? c_10_from_00 : c_10_from_01;
wire [1:0] src_10 = (c_10_from_00 <= c_10_from_01) ? 2'b00 : 2'b01;

wire [4:0] sum01 = (c_01_from_10 <= c_01_from_11) ? c_01_from_10 : c_01_from_11;
wire [1:0] src_01 = (c_01_from_10 <= c_01_from_11) ? 2'b10 : 2'b11;

```

```

    wire [4:0] sum11 = (c_11_from_10 <= c_11_from_11) ? c_11_from_10 :
c_11_from_11;
    wire [1:0] src_11 = (c_11_from_10 <= c_11_from_11) ? 2'b10 : 2'b11;

    // BƯỚC 3: TÌM BEST NODE (thứ tự ưu tiên nếu pm bằng nhau: 00 > 10 > 01
> 11)
    wire [4:0] min_00_10 = (sum00 <= sum10) ? sum00 : sum10;
    wire [1:0] node_00_10 = (sum00 <= sum10) ? 2'b00 : 2'b10;

    wire [4:0] min_01_11 = (sum01 <= sum11) ? sum01 : sum11;
    wire [1:0] node_01_11 = (sum01 <= sum11) ? 2'b01 : 2'b11;

    wire [1:0] best_node = (min_00_10 <= min_01_11) ? node_00_10 :
node_01_11;

    // BƯỚC 4: LOGIC TUẦN TỰ, cập nhật thanh ghi
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            pm_00    <= 5'd0;
            pm_01    <= 5'd0;
            pm_10    <= 5'd0;
            pm_11    <= 5'd0;
            prev_st_00 <= 2'b00;
            prev_st_01 <= 2'b00;
            prev_st_10 <= 2'b00;
            prev_st_11 <= 2'b00;
            slt_node  <= 2'b00;
            o_valid   <= 1'b0;
            o_sof     <= 1'b0;
        end else if (i_valid) begin
            // Chỉ chốt dữ liệu khi cờ i_valid báo có dữ liệu
            pm_00    <= sum00;
            pm_01    <= sum01;

```

```

    pm_10    <= sum10;
    pm_11    <= sum11;

    prev_st_00 <= src_00;
    prev_st_10 <= src_10;
    prev_st_01 <= src_01;
    prev_st_11 <= src_11;

    slt_node  <= best_node;

    // Dịch trể cờ sang module sau 1 nhịp ck (vì tính toán mất 1 clk)
    o_valid   <= 1'b1;
    o_sof     <= i_sof;
end else begin
    // Khi không có dữ liệu hợp lệ, tắt cờ o_valid
    // Các dữ liệu khác giữ nguyên
    pm_00     <= pm_00;
    pm_01     <= pm_01;
    pm_10     <= pm_10;
    pm_11     <= pm_11;
    prev_st_00 <= prev_st_00;
    prev_st_01 <= prev_st_01;
    prev_st_10 <= prev_st_10;
    prev_st_11 <= prev_st_11;
    slt_node  <= slt_node;
    o_valid   <= 1'b0;
    o_sof     <= 1'b0;
end
end
endmodule

```


4. Module memory:

```
module memory (  
    input wire clk,  
    input wire rst_n,  
  
    // Tín hiệu Hand-shake từ module ACS  
    input wire i_valid,  
    input wire i_sof,  
  
    // Input vào từ module ACS  
    input wire [1:0] prev_st_00,  
    input wire [1:0] prev_st_01,  
    input wire [1:0] prev_st_10,  
    input wire [1:0] prev_st_11,  
  
    // Output cấp cho module Traceback  
    output wire [1:0] bck_prev_st_00,  
    output wire [1:0] bck_prev_st_01,  
    output wire [1:0] bck_prev_st_10,  
    output wire [1:0] bck_prev_st_11,  
  
    // Cờ báo hiệu cho Traceback biết Memory đã chuyển sang pha Đọc  
    output reg o_memory_done  
);  
  
    // Sử dụng 1 mảng 2 chiều 4x8, mỗi ô nhớ 2 bit làm mảng nhớ  
    // [0:3]: 4 hàng tương ứng 4 trạng thái  
    // [0:7]: 8 cột tương ứng 8 thời điểm  
    reg [1:0] trellis_diagr [0:3][0:7];  
  
    // Các biến đếm nội bộ  
    reg [2:0] count; // Biến đếm ghi  
    reg [2:0] trace; // Biến đếm đọc  
    reg read_mode; // Cờ phân biệt Ghi/Đọc
```

```

// LOGIC TUẦN TỰ: ĐIỀU KHIỂN GHI / ĐỌC
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        count        <= 3'd0;
        trace        <= 3'd7;
        read_mode    <= 1'b0;
        o_memory_done <= 1'b0;
    end
    else begin
        // GIAI ĐOẠN GHI
        if (i_valid && !read_mode) begin
            if (i_sof) begin
                // Nếu có cờ sof -> Ghi dữ liệu vào cột 0 và reset count
                trellis_diagr[0][0] <= prev_st_00;
                trellis_diagr[1][0] <= prev_st_01;
                trellis_diagr[2][0] <= prev_st_10;
                trellis_diagr[3][0] <= prev_st_11;
                count <= 3'd1;
            end
            else begin
                // Các cột tiếp theo -> Ghi vào cột ở địa chỉ count
                trellis_diagr[0][count] <= prev_st_00;
                trellis_diagr[1][count] <= prev_st_01;
                trellis_diagr[2][count] <= prev_st_10;
                trellis_diagr[3][count] <= prev_st_11;

                // Nếu vừa ghi xong cột cuối cùng (cột 7)
                if (count == 3'd7) begin
                    read_mode    <= 1'b1; // Chuyển sang Giai đoạn Đọc
                    trace        <= 3'd7; // Khởi tạo con trỏ trace ở cột 7
                    o_memory_done <= 1'b1; // Bật cờ hand-shake tới khối Traceback
                    count        <= 4'd0; // Reset count chuẩn bị cho chuỗi mới
                end
            end
        end
    end
end

```

```

        count <= count + 1'b1; // Tăng biến đếm ghi
    end
end
end

// GIAI ĐOẠN ĐỌC

else if (read_mode) begin
    // Tất cả done
    o_memory_done <= 1'b0;

    // Giảm dần biến trace theo từng chu kỳ
    if (trace > 3'd0) begin
        trace <= trace - 1'b1;
    end
    else begin
        // Khi trace về 0, kết thúc chế độ đọc
        read_mode <= 1'b0;
    end
end
end
end

// Xuất đồng thời 4 trạng thái tại cột địa chỉ trace
assign bck_prev_st_00 = read_mode ? trellis_diagr[0][trace] : 2'b00;
assign bck_prev_st_01 = read_mode ? trellis_diagr[1][trace] : 2'b00;
assign bck_prev_st_10 = read_mode ? trellis_diagr[2][trace] : 2'b00;
assign bck_prev_st_11 = read_mode ? trellis_diagr[3][trace] : 2'b00;
endmodule

```

5. Module traceback:

```
module traceback (  
    input wire clk,  
    input wire rst_n,  
    input wire i_memory_done, // Hand-shake từ memory  
    input wire [1:0] slt_node, // Điểm bắt đầu được gửi từ ACS  
    input wire [1:0] bck_prev_st_00,  
    input wire [1:0] bck_prev_st_01,  
    input wire [1:0] bck_prev_st_10,  
    input wire [1:0] bck_prev_st_11,  
    output reg [7:0] o_data, // Dữ liệu đã được giải mã hoàn thành  
    output reg o_done // Cờ báo hoàn tất giải mã  
);  
    // Khai báo các biến nội bộ  
    reg [2:0] count; // Đếm 8 chu kỳ  
    reg [1:0] current_node; // Trạng thái hiện tại đang xét  
    reg [7:0] temp_data; // Thanh ghi dịch chứa chuỗi kết quả tạm  
    reg tracing; // Cờ báo hiệu mạch đang chạy  
  
    // Bộ MUX chọn trạng thái liên trước  
    reg [1:0] target_node;  
    reg [1:0] next_node;  
    always @(i_memory_done, slt_node, current_node, bck_prev_st_00,  
bck_prev_st_01, bck_prev_st_10, bck_prev_st_11) begin  
        // Chu kì đầu tiên chọn node theo slt_node được gửi từ ACS  
        if (i_memory_done == 1'b1) begin  
            target_node = slt_node;  
        end else begin  
            target_node = current_node;  
        end  
  
        // Chọn node tiếp theo  
        case (target_node)  
            2'b00: next_node = bck_prev_st_00;
```

```

        2'b01: next_node = bck_prev_st_01;
        2'b10: next_node = bck_prev_st_10;
        default: next_node = bck_prev_st_11;
    endcase
end

// Logic truy vết
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        count      <= 3'd0;
        current_node <= 2'b00;
        temp_data   <= 8'd0;
        tracing     <= 1'b0;
        o_data      <= 8'd0;
        o_done      <= 1'b0;
    end else begin
        // Nhận tín hiệu hand-shake từ memory
        if (i_memory_done) begin
            tracing <= 1'b1;
            count  <= 3'd0; // Đã xử lý xong nhịp đầu tiên

            // B1: Lấy MSB của node đang đứng làm bit giải mã và dịch nó vào
            thanh ghi
            // (theo quy tắc của bộ mã hóa có giải thích trong spec <''))
            temp_data <= {target_node[1], temp_data[7:1]};

            // B2: Lùi, ghi dấu chân tiếp theo vào current_node
            current_node <= next_node;
            o_done      <= 1'b0;
        end

        // Quá trình truy vết các chu kỳ tiếp theo
        else if (tracing) begin
            if (count < 3'd7) begin

```

```

        // Lấy MSB của current_node làm bit giải mã và dịch vào
        temp_data  <= {target_node[1], temp_data[7:1]};

        // Lùi bước
        current_node <= next_node;

        // Tăng biến đếm
        count      <= count + 1'b1;
    end
    // Hoàn thành truy vết
    else begin
        tracing    <= 1'b0; // Dừng truy vết
        o_data     <= temp_data; // Gửi dữ liệu ra output
        o_done     <= 1'b1; // Bật cờ done
        count      <= 3'd0; // Reset biến đếm
    end
end
end

// Tắt cờ o_done sau 1 nhịp clk
else if (o_done) begin
    o_done <= 1'b0;
end
end
end
endmodule

```

6. Module viterbi_top:

```
module viterbi_top (  
    input wire clk,  
    input wire rst_n,  
    input wire i_start,  
    input wire [15:0] i_data, // Dữ liệu mã hóa đầu vào  
    output wire [7:0] o_data, // Dữ liệu giải mã đầu ra  
    output wire o_done // Cờ báo hiệu hoàn tất  
);  
    // Khai báo dây nối nội bộ  
    // extract_bit -> branch_metric  
    wire [1:0] w_rx;  
    wire w_valid_1;  
    wire w_sof_1;  
    // branch_metric -> add_comp_slt  
    wire [1:0] w_hd_00, w_hd_01, w_hd_10, w_hd_11;  
    wire w_valid_2;  
    wire w_sof_2;  
    // add_comp_slt -> memory & traceback  
    wire [1:0] w_prev_st_00, w_prev_st_01, w_prev_st_10, w_prev_st_11;  
    wire [1:0] w_slt_node;  
    wire w_valid_3;  
    wire w_sof_3;  
    // memory -> traceback  
    wire [1:0] w_bck_prev_st_00, w_bck_prev_st_01, w_bck_prev_st_10,  
w_bck_prev_st_11;  
    wire w_memory_done;  
    // Khởi tạo và ghép nối các module con  
    // Module 1: extract_bit  
    extract_bit u_extract(  
        .clk      (    clk      ),  
        .rst_n    (    rst_n    ),  
        .i_start  (    i_start  ),  
        .i_data   (    i_data   ),
```

```

.o_rx      (    w_rx      ),
.o_valid   (    w_valid_1  ),
.o_sof     (    w_sof_1   )
);
// Module 2: branch_metric
branch_metric u_bm (
    .rst_n   (    rst_n    ),
    .i_valid (    w_valid_1 ),
    .i_sof   (    w_sof_1  ),
    .i_rx    (    w_rx     ),
    .hd_00   (    w_hd_00  ),
    .hd_01   (    w_hd_01  ),
    .hd_10   (    w_hd_10  ),
    .hd_11   (    w_hd_11  ),
    .o_valid (    w_valid_2 ),
    .o_sof   (    w_sof_2  )
);
// Module 3: add_comp_slt
add_comp_slt u_acs (
    .clk      (    clk      ),
    .rst_n    (    rst_n    ),
    .i_valid  (    w_valid_2 ),
    .i_sof    (    w_sof_2  ),
    .hd_00    (    w_hd_00  ),
    .hd_01    (    w_hd_01  ),
    .hd_10    (    w_hd_10  ),
    .hd_11    (    w_hd_11  ),
    .prev_st_00 (    w_prev_st_00 ),
    .prev_st_01 (    w_prev_st_01 ),
    .prev_st_10 (    w_prev_st_10 ),
    .prev_st_11 (    w_prev_st_11 ),
    .slt_node  (    w_slt_node ),
    .o_valid   (    w_valid_3 ),
    .o_sof     (    w_sof_3  )
);

```



```

);
// Module 4: memory
memory u_memory (
    .clk      (    clk      ),
    .rst_n    (    rst_n    ),
    .i_valid  (    w_valid_3    ),
    .i_sof    (    w_sof_3    ),
    .prev_st_00 (    w_prev_st_00    ),
    .prev_st_01 (    w_prev_st_01    ),
    .prev_st_10 (    w_prev_st_10    ),
    .prev_st_11 (    w_prev_st_11    ),
    .bck_prev_st_00 (    w_bck_prev_st_00    ),
    .bck_prev_st_01 (    w_bck_prev_st_01    ),
    .bck_prev_st_10 (    w_bck_prev_st_10    ),
    .bck_prev_st_11 (    w_bck_prev_st_11    ),
    .o_memory_done (    w_memory_done    )
);
// Module 5: traceback
traceback u_traceback(
    .clk      (    clk      ),
    .rst_n    (    rst_n    ),
    .i_memory_done (    w_memory_done    ),
    .slt_node  (    w_slt_node    ),
    .bck_prev_st_00 (    w_bck_prev_st_00    ),
    .bck_prev_st_01 (    w_bck_prev_st_01    ),
    .bck_prev_st_10 (    w_bck_prev_st_10    ),
    .bck_prev_st_11 (    w_bck_prev_st_11    ),
    .o_data    (    o_data    ),
    .o_done    (    o_done    )
);
endmodule

```